

Công nghệ phần mềm

Chương 04: Thiết kế phần mềm



Giảng viên: Lê Thị Hoàng Anh

Email: anh1th@nuce.edu.vn



Nội dung

1. Tổng quan về thiết kế phần mềm

- Thiết kế phần mềm trong kỹ nghệ phần mềm
- Các giai đoạn trong thiết kế phần mềm
- Quá trình thiết kế
- Phương pháp thiết kế

2. Thiết kế hướng đối tượng

3. Các nguyên tắc thiết kế

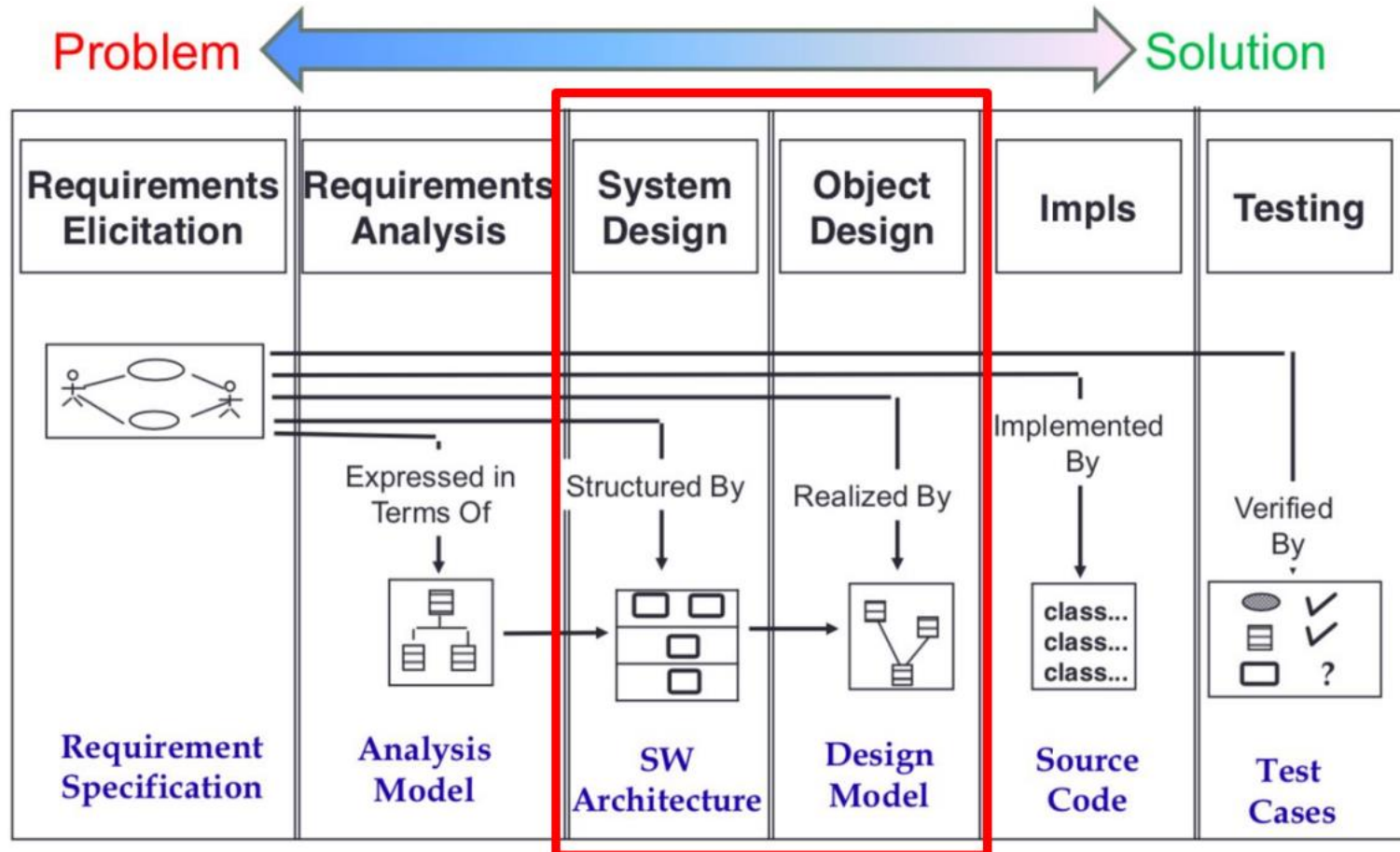
4. Các mẫu thiết kế

5. Thiết kế giao diện người sử dụng

6. Thiết kế dữ liệu



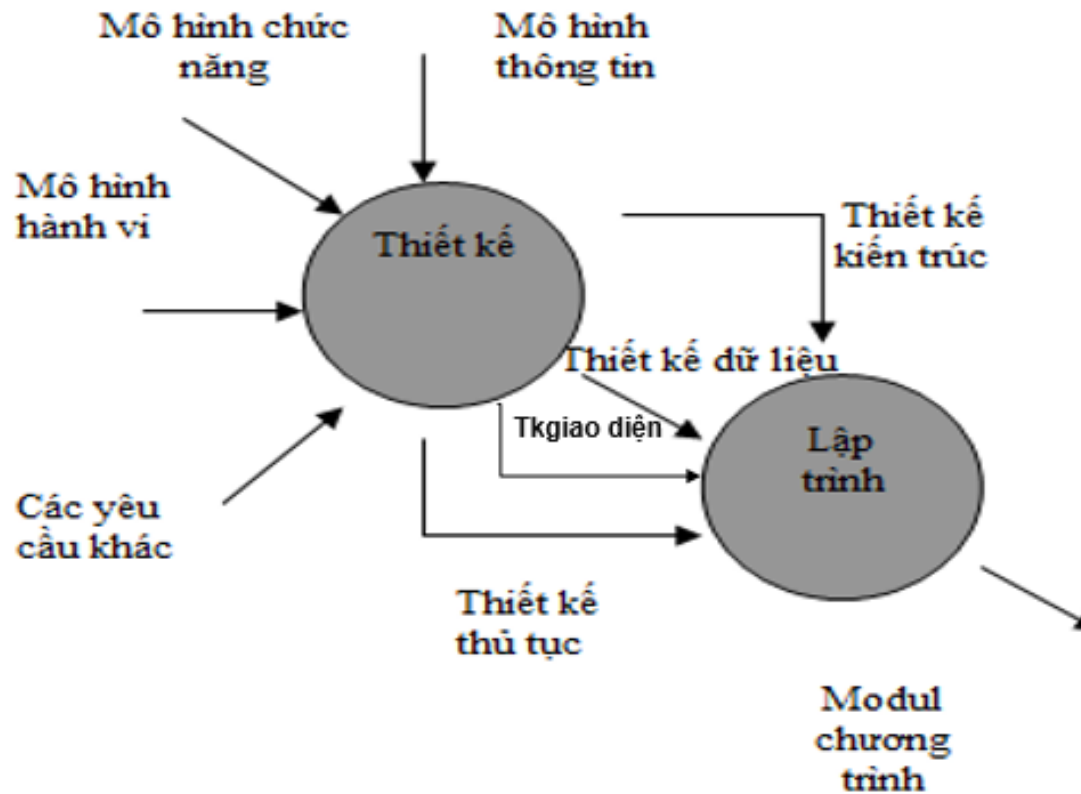
Tổng quan về thiết kế phần mềm





Tổng quan về thiết kế phần mềm

- Thiết kế phần mềm trong kỹ nghệ phần mềm:





Tổng quan về thiết kế phần mềm

- Thiết kế là chuyển bản đặc tả yêu cầu thành mô tả thiết kế đủ chi tiết để người lập trình có thể chuyển thành chương trình với 1 ngôn ngữ lập trình, vận hành đáp ứng được yêu cầu đặt ra.
- Bao gồm:
 - Thiết kế hệ thống: Thiết kế cấu trúc ngoài của phần mềm.
 - Thiết kế chương trình: Thiết kế chi tiết cấu trúc bên trong của phần mềm. Thiết kế tính năng từng môđun và giao diện tương ứng



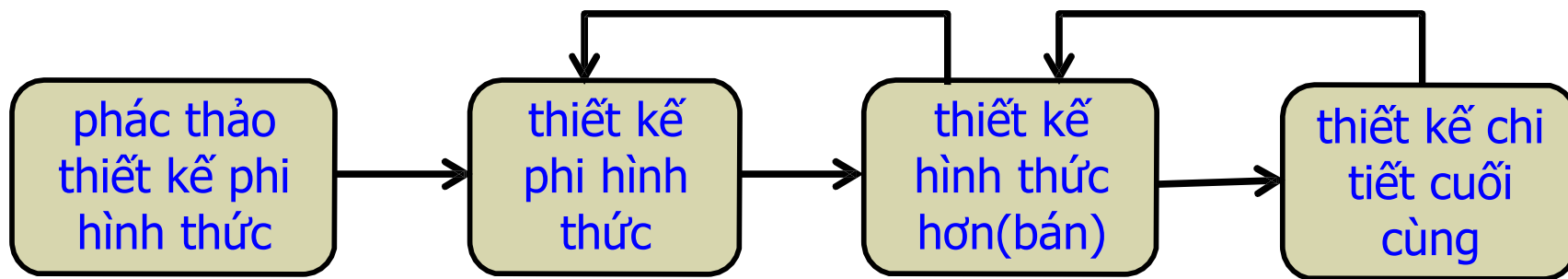
Tổng quan về thiết kế phần mềm

- Các giai đoạn trong thiết kế phần mềm:
Thiết kế phần mềm trải qua ba giai đoạn
 - Nghiên cứu và tìm hiểu các kết quả phân tích và đặc tả yêu cầu phần mềm ở công đoạn trước.
 - Lựa chọn các giải pháp thiết kế. Chọn giải pháp dựa vào kinh nghiệm của người thiết kế.
 - Triển khai công việc thiết kế.



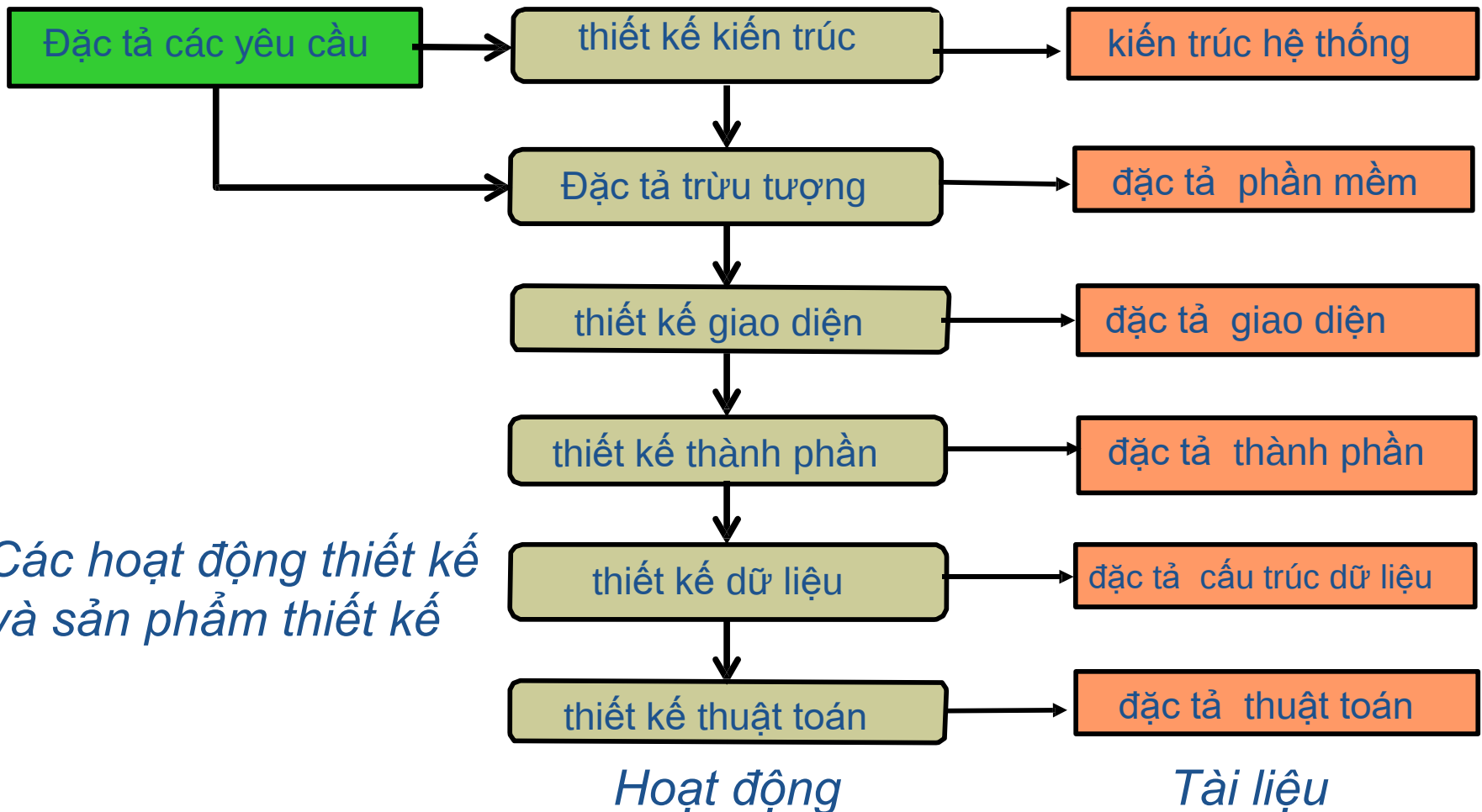
Tổng quan về thiết kế phần mềm

- Quá trình thiết kế: Là quá trình tăng cường hình thức hóa và luôn quay lại các thiết kế đúng đắn và ít hình thức hóa hơn trước đó để kiểm tra và hoàn chỉnh.





Quá trình thiết kế phần mềm





Tổng quan về thiết kế phần mềm

- Quá trình thiết kế: Bao gồm các hoạt động
 - Thiết kế kiến trúc:
 - Là giai đoạn đầu của quá trình thiết kế hệ thống
 - Là quá trình xác định:
 - Những hệ thống con hoặc mô đun
 - Và một khung làm việc giúp kiểm soát và giao tiếp giữa các hệ thống con/mô đun này.
 - Đặc tả trừu tượng:
 - Đối với mỗi hệ con, một đặc tả trừu tượng
 - Các dịch vụ mà nó cung cấp
 - Và các ràng buộc tuân theo phải cung cấp



Tổng quan về thiết kế phần mềm

- Quá trình thiết kế (tiếp):
 - Thiết kế giao diện:
 - Là thiết kế giao diện của từng hệ con và với các hệ con khác.
 - Thiết kế thành phần:
 - Các dịch vụ được cung cấp bởi hệ con được phân chia qua các thành phần hợp thành của hệ con đó
 - Thiết kế dữ liệu:
 - Thiết kế CSDL được dùng trong việc thực hiện hệ thống ở các mức chi tiết đặc tả khác nhau.
 - Thiết kế thuật toán:
 - Các thuật toán được thiết kế chi tiết và đặc tả



Tổng quan về thiết kế phần mềm

- Trong quá trình thiết kế, các hoạt động này được lặp lại cho các hệ con cho đến khi rõ ràng, đủ chi tiết để có thể minh định.
- Phương pháp tiếp cận thường dùng trong thiết kế là tiếp cận từ trên xuống (top-down)
- Các bước tiến hành thiết kế theo quan điểm quản lý dự án:
 1. Thiết kế sơ bộ
 2. Thiết kế chi tiết



Tổng quan về thiết kế phần mềm

- Phương pháp thiết kế:
 - Phương pháp hướng cấu trúc
 - Phương pháp hướng đối tượng
 - Phương pháp hướng khía cạnh (AOSD)
- Các phương pháp thiết kế trên thường trợ giúp một số cách nhìn nhận hệ thống:
 - Nhìn nhận cấu trúc
 - Nhìn nhận quan hệ thực thể
 - Nhìn nhận dòng dữ liệu



Tổng quan về thiết kế phần mềm

- Các đặc trưng của phương pháp thiết kế:
 1. Một cơ chế để chuyển hóa từ biểu diễn miền thông tin thành biểu diễn thiết kế
 2. Một bộ ký pháp để biểu diễn các thành phần chức năng và giao diện giữa chúng
 3. Cách trực cảm để làm mịn phân hoạch
 4. Các hướng dẫn về định giá chất lượng



Tổng quan về thiết kế phần mềm

- Phương pháp cấu trúc là phương pháp đầu tiên ra đời vào những đầu năm thập kỷ 1970
- Những năm thập kỷ 1990 phương pháp hướng đối tượng trở thành trào lưu ra đời.
- Thay vì phương pháp dựa vào chức năng như phương pháp cấu trúc, phương pháp hướng đối tượng lại dựa vào các thực thể (các đối tượng).
- Khái niệm hướng đối tượng được xây dựng trên nền tảng hướng cấu trúc và sự trừu tượng hóa dữ liệu.



Tổng quan về thiết kế phần mềm

- Hạn chế của các phương pháp:
 - Phương pháp hướng cấu trúc: Lộ rõ yếu điểm khi
 - Độ phức tạp của một chương trình tăng lên
 - Cấu trúc dữ liệu phức tạp lên
 - Sử dụng dữ liệu dùng chung là khó cho việc làm theo nhóm
 - Phương pháp hướng đối tượng:
 - Chỉ dừng lại ở mức tĩnh (Sinh – Hoạt – Tử)
 - Thiếu tính khả mở, tiến hóa



Tổng quan về thiết kế phần mềm

- Hạn chế của các phương pháp trên:
 - Một hệ thống thực tế là sự đan xen giữa các yêu cầu. Một yêu cầu nghiệp vụ thường được bổ sung thêm các yêu cầu khác, gây nên sự chồng chéo giữa các yêu cầu. Từ đó hệ thống thiết kế theo hướng đối tượng sẽ khó bảo trì, phát triển. Khi bổ sung thêm một nghiệp vụ phải dẫn đến thay đổi tất cả các yêu cầu nghiệp vụ liên quan.



Tổng quan về thiết kế phần mềm

- Phương pháp hướng khía cạnh:
 - Được xây dựng dựa trên phương pháp hướng cấu trúc và hướng đối tượng. Bổ sung thêm các khái niệm và cấu trúc để module hóa các concern đan xen.
 - Khác nhau cơ bản của hướng khía cạnh là cách quản lý các nhiệm vụ đan xen (*cross-cutting concerns*) và các nhiệm vụ thành phần (*core concerns*)



Tổng quan về thiết kế phần mềm

- Bất kể phương pháp thiết kế nào được dùng. Kiến trúc sư phần mềm cũng phải áp dụng một tập các khái niệm nền tảng cho thiết kế bao gồm như sau:
 - Trừu tượng hóa: Trừu tượng hóa dữ liệu, thủ tục, điều khiển.
 - Làm mịn: Chi tiết hóa các trừu tượng theo ý đồ.
 - Tính môđun: Phân chia dữ liệu và chức năng
 - Kiến trúc: Cấu trúc tổng thể của phần mềm
 - Thủ tục: Thuật toán để thực hiện chức năng
 - Che dấu thông tin: Điều khiển bằng giao diện



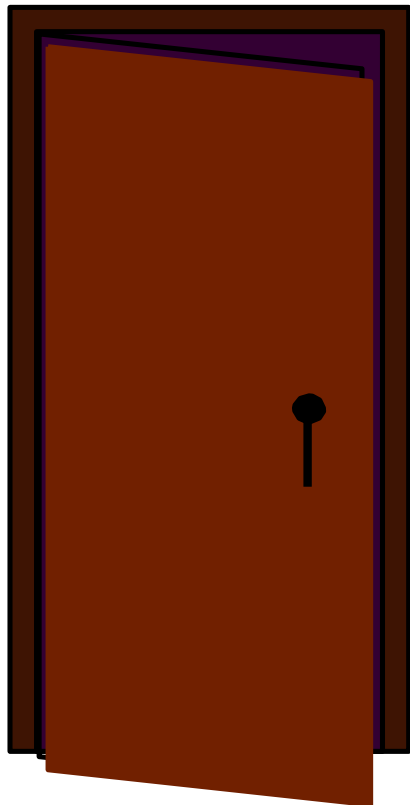
Trừu tượng hóa

- Là quá trình ánh xạ một sự vật/hiện tượng của thế giới thực thành một khái niệm logic.
- Giảm thiểu tinh lọc các chi tiết nhằm tập trung vào một số khái niệm/vấn đề quan tâm tại một thời điểm.
- Có nhiều mức trừu tượng khác nhau:
 - Cho phép con người tập trung vào giải quyết vấn đề mà không cần bận tâm đến chi tiết.
 - Biểu diễn vấn đề bằng một cấu trúc tự nhiên.



Trừu tượng hóa

1. Trừu tượng dữ liệu:



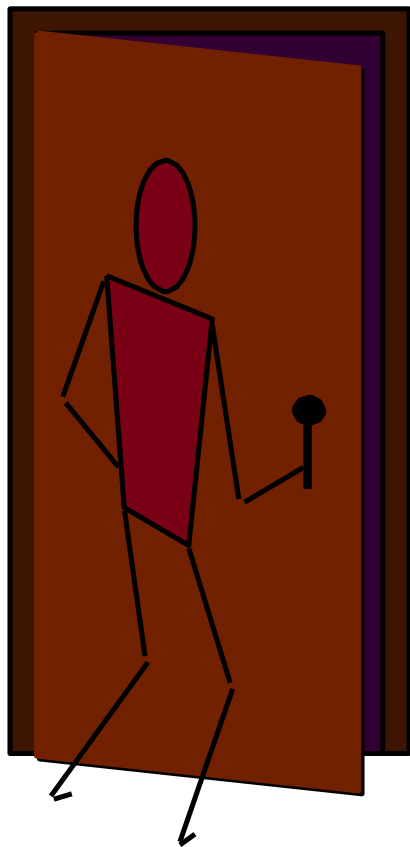
Cửa

mã số: 256AD loại: *cửa mở kéo*
hướng mở: *ra bên trái*
cao: *2.3 m*
rộng: *0.85 m*
trọng lượng: *120*
màu: *nâu cánh dán*



Trừu tượng hóa

2. Trừu tượng thủ tục :



Mở cửa

Mô tả chi tiết quá trình vào phòng qua cửa



Làm mịn từng bước

Mở cửa

Bước đến gần cửa
Đưa chìa khóa vào ổ xoay

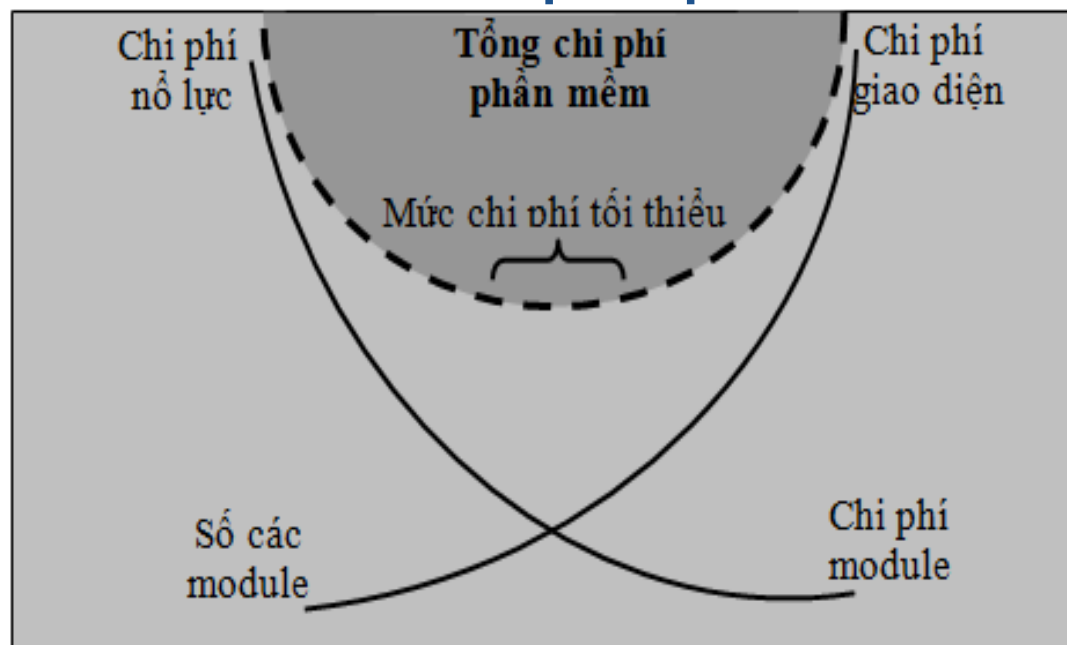
Mở cửa →
Bước qua vào phòng
Đóng cửa lại

Lặp lại cho đến khi chốt khóa bật ra
Nếu chốt không mở thì
Rút khóa ra, tìm chìa khác phù
hợp, cắm vào ổ khóa, tiếp tục
xoay cho đến khi mở được



Tính mô đun

- Phần mềm được chia thành các thành phần có tên riêng biệt và xác định được địa chỉ, gọi là các mô đun.
- Tính mô đun và chi phí phần mềm:





Che giấu thông tin

- Sử dụng môđun thông qua các giao diện, danh sách tham số và giá trị trả lại.
- Không cần biết cách thức cài đặt của nó:
 - Thuật toán
 - Cấu trúc dữ liệu
 - Giao diện ngoại lai (môđun thứ cấp, thiết bị vào ra)
 - Tài nguyên hệ thống



Che dấu thông tin

- Lý do che dấu thông tin:
 - Giảm hiệu ứng phụ khi sửa đổi mô đun
 - Giảm tác động của thiết kế tổng thể lên thiết kế cục bộ
 - Nhấn mạnh trao đổi thông tin thông qua giao diện
 - Loại bỏ việc sử dụng dữ liệu dùng chung
 - Hướng tới sự đóng gói chức năng là thuộc tính của một thiết kế tốt

➡ **Tạo ra các sản phẩm phần mềm tốt hơn!**



Nội dung

→ 2. Thiết kế hướng đối tượng

- Cách tiếp cận hướng đối tượng
- Đặc trưng của thiết kế hướng đối tượng
- Các ưu nhược điểm của thiết kế hướng đối tượng
- Phân biệt giữa thiết kế hướng đối tượng và lập trình hướng đối tượng
- Sử dụng các loại biểu đồ mô tả cấu trúc của phần mềm
- Phương pháp xây dựng biểu đồ class
- Sử dụng các loại biểu đồ mô tả tương tác của phần mềm



Thiết kế hướng đối tượng

- Các thuật ngữ trong phương pháp hướng đối tượng:
 - OOA liên quan đến việc phát triển một mô hình đối tượng của miền ứng dụng.
 - OOD liên quan đến việc mô tả tất cả các đối tượng và sự tương tác giữa chúng trong hệ thống để có thể cài đặt được các yêu cầu.
 - OOP liên quan đến việc thực hiện cài đặt một bản thiết kế bằng cách sử dụng một ngôn ngữ lập trình OO như Java hay .NET, C++ ...



Cách tiếp cận hướng đối tượng

- Bản chất của thiết kế hướng đối tượng là khả năng xây dựng ba khai niệm thiết kế phần mềm quan trọng:
 - Trừu tượng
 - Che dấu thông tin
 - Modul

Đặc trưng của thiết kế hướng đối tượng

- Không có vùng dữ liệu dùng chung.
 - Các đối tượng liên lạc với nhau bằng cách trao đổi thông báo chứ không phải các biến dùng chung.
- Các đối tượng là các thực thể độc lập, dễ thay đổi
- Các đối tượng có thể phân tán và có thể hành động tuần tự hoặc song song

Các ưu điểm, nhược điểm của TKHĐT

- Ưu điểm:
 - Dễ bảo trì vì các đối tượng là độc lập
 - Các đối tượng là các thành phần dùng lại được. Một thiết kế có thể dùng lại các đối tượng đã được thiết kế trong các bản thiết kế trước đó.
 - Đạt tính dễ hiểu trong thiết kế nhờ:
 - Các lớp trong hệ thống tương ứng với các thực thể ngoài đời thực.
 - Các tương tác điều khiển tương ứng với các hoạt động của đối tượng ngoài thế giới thực.

Các ưu điểm, nhược điểm của TKHĐT

- Nhược điểm:
 - Việc minh định các đối tượng trong hệ thống là khó khăn.
 - Cách nhìn tự nhiên là cách nhìn hướng chức năng, do vậy việc thích nghi với hướng đối tượng cho người mới bắt đầu thường khó khăn hơn.



Phân biệt giữa TKHĐT và LTHĐT

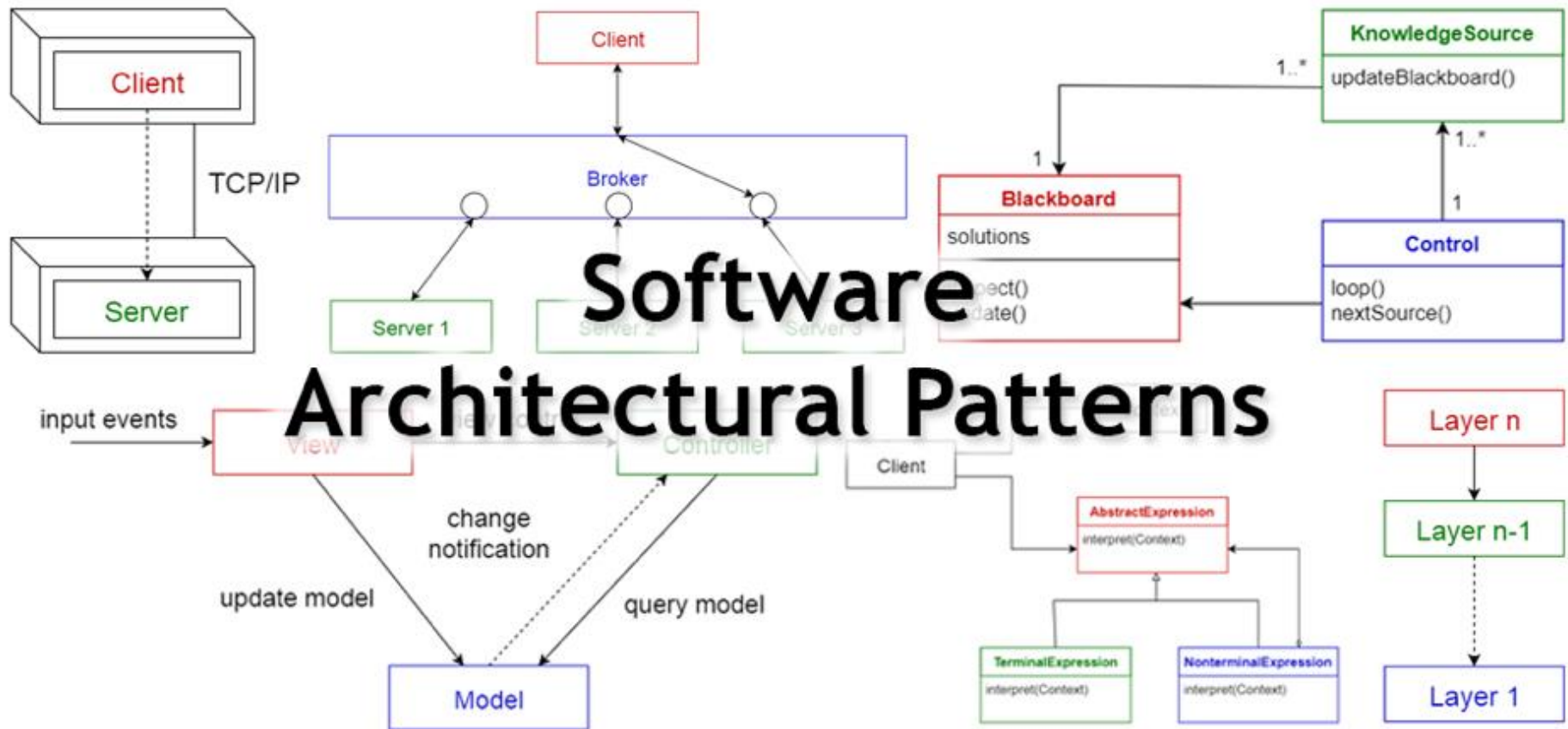
- Lập trình hướng đối tượng là việc lập trình sử dụng một ngôn ngữ OO.
- Thiết kế hướng đối tượng là một chiến lược thiết kế không phụ thuộc vào ngôn ngữ cụ thể.
- Có thể thực hiện bản thiết kế trên bằng cả ngôn ngữ OO hoặc ngôn ngữ không hỗ trợ OO (tuy khó khăn hơn).

Sử dụng các loại biểu đồ mô tả cấu trúc của phần mềm

- Static Model: mô tả cấu trúc tĩnh của hệ thống. Bao gồm một số các bản vẽ như biểu đồ lớp, đối tượng ...
- Bài học chỉ đề cập đến:
 - Mô hình kiến trúc phần mềm
 - Biểu đồ lớp (Class diagram)
 - Biểu đồ đối tượng (Object diagram)
 - Biểu đồ thành phần (Component diagram)
 - Biểu đồ triển khai (Deployment diagram)



Mô hình kiến trúc phần mềm





Mô hình kiến trúc phần mềm

- Kiến trúc phần mềm là một hệ thống cấu trúc các thành phần cần thiết, đặc điểm, tính chất cũng như mối quan hệ giữa chúng.
- Thiết kế kiến trúc phần mềm:
 - Là giai đoạn đầu của quá trình thiết kế hệ thống
 - Là quá trình mô tả và phát triển hệ thống nhằm xác định:
 - Những hệ thống con hoặc mô-đun và
 - một khung làm việc giúp kiểm soát và giao tiếp giữa các hệ thống con/mô-đun này



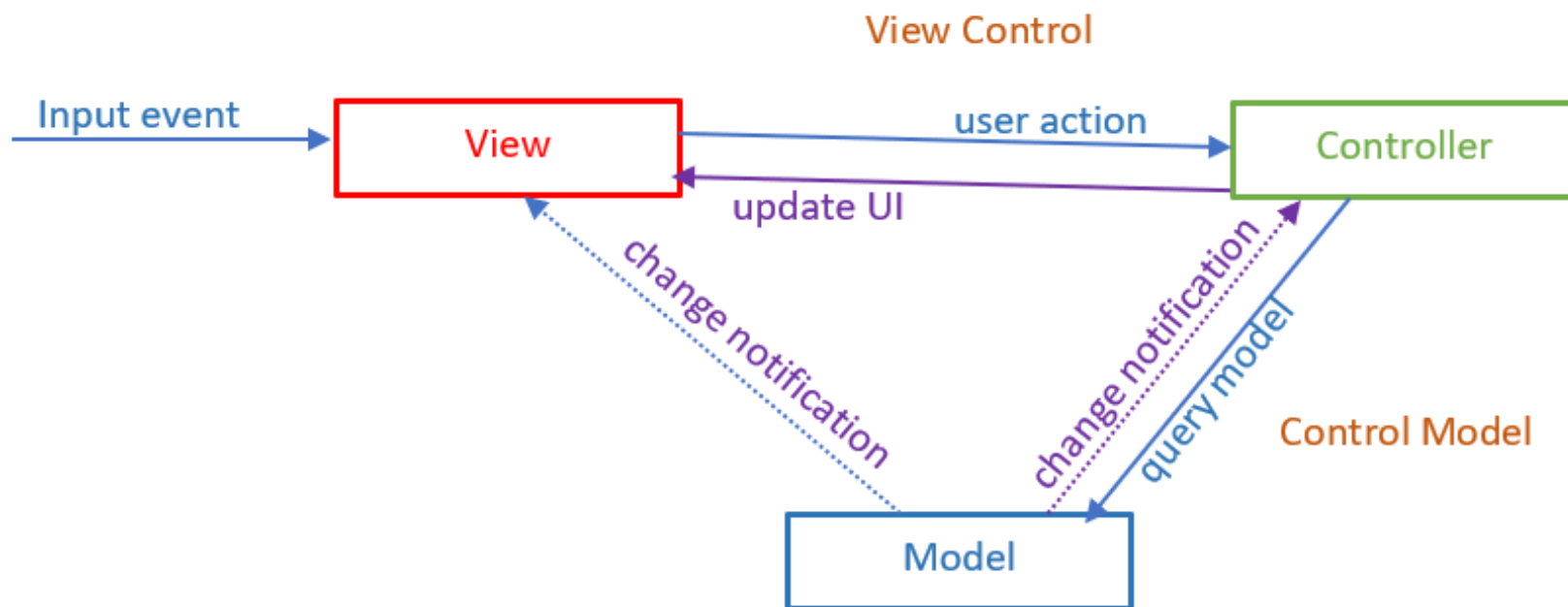
Mô hình kiến trúc phần mềm

- Một số mẫu mô hình kiến trúc phần mềm phổ biến:
 - Kiến trúc MVC (**Model-view-controller pattern**)
 - Kiến trúc phân tầng (**Layered pattern**)
 - Kiến trúc Khách – chủ (**Client-server pattern**)
 - Kiến trúc ống – lọc (**Pipe-filter pattern**)
 - Kiến trúc chủ - thợ (**Master-slave pattern**)
 - Kiến trúc dựa trên sự kiện (**Event-bus pattern**)
 - Kiến trúc kho (Repository Architecture)
 - Kiến trúc bảng đen (Blackboard Architecture)
 - ...



Mô hình kiến trúc phần mềm

- MVC

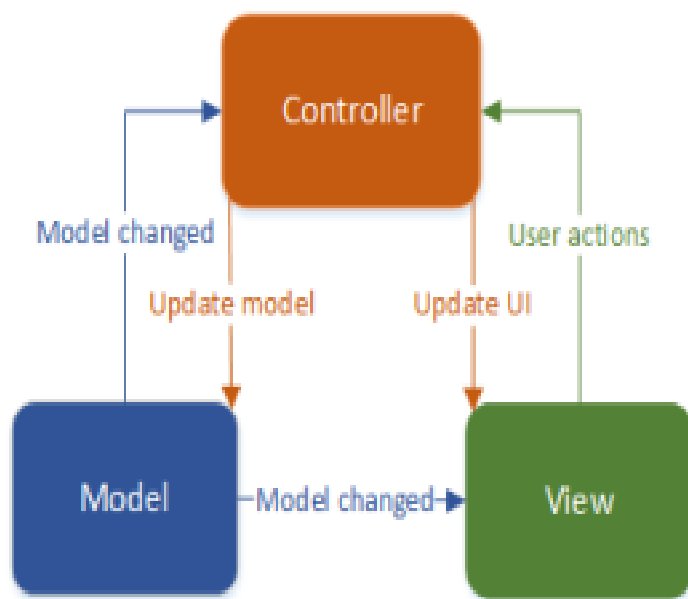




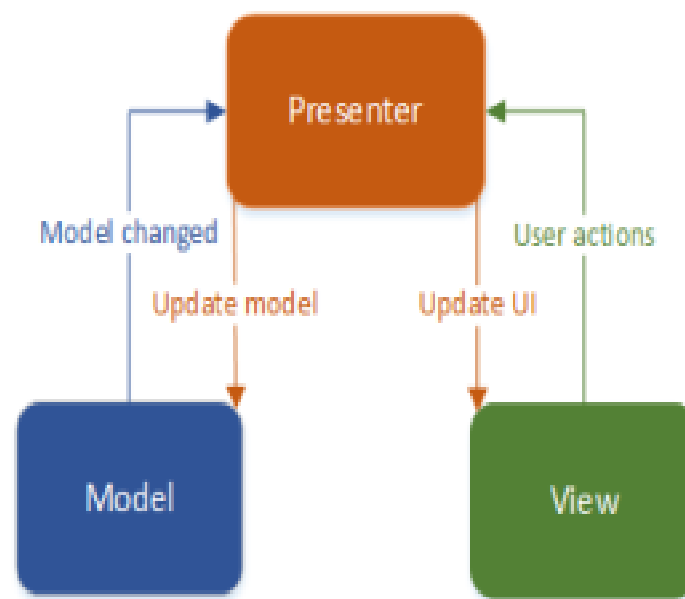
So sánh MVP và MVC

- Hai mô hình xử lý

MVC



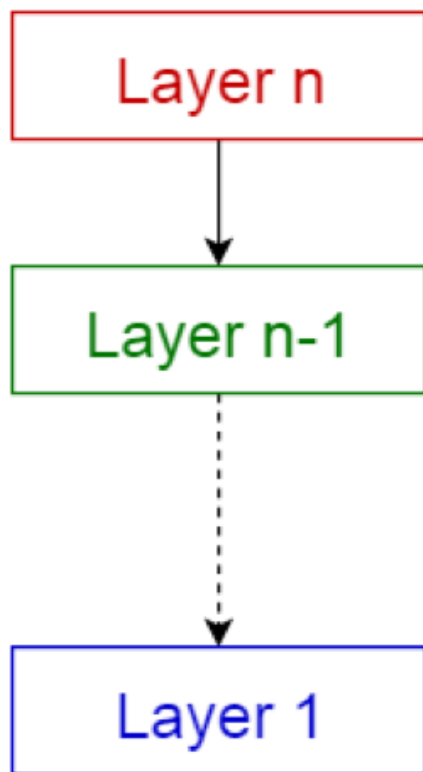
MVP



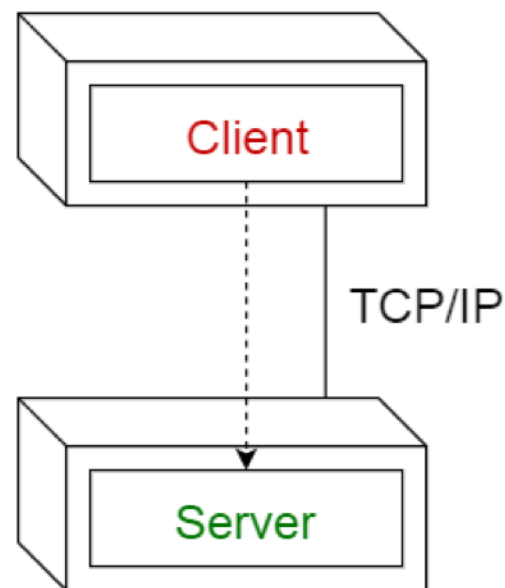


Mô hình kiến trúc phần mềm

Phân tầng



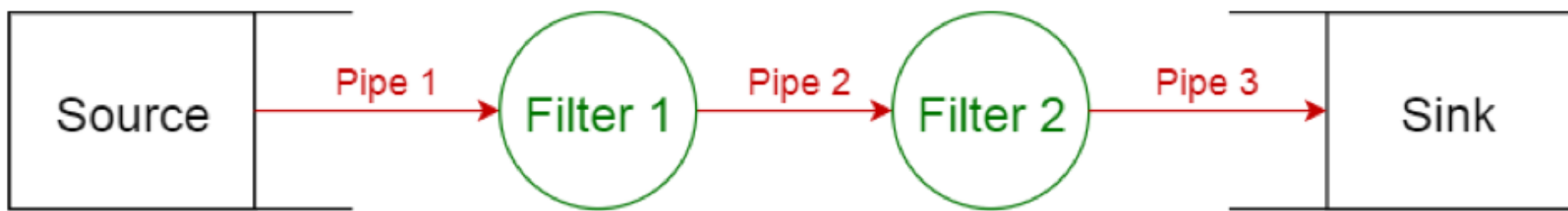
Client – Server



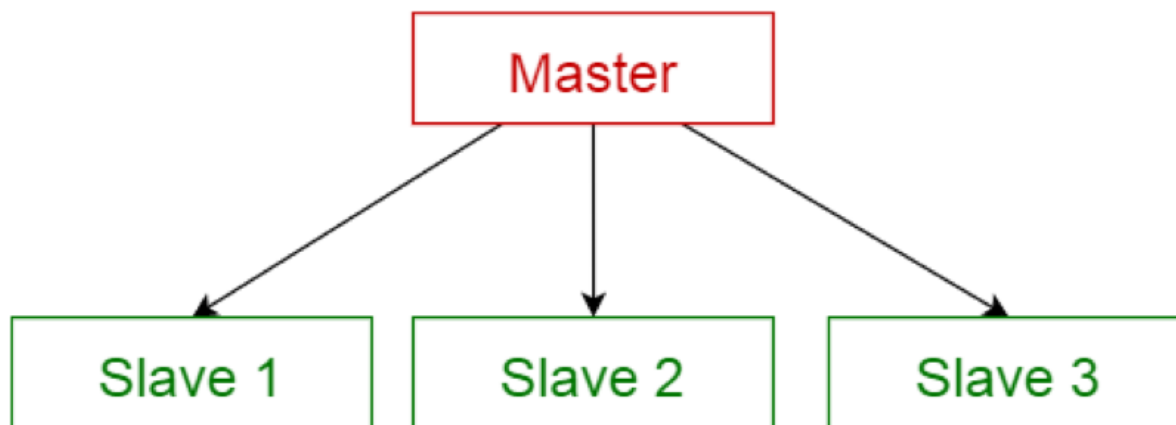


Mô hình kiến trúc phần mềm

- Ống - lọc



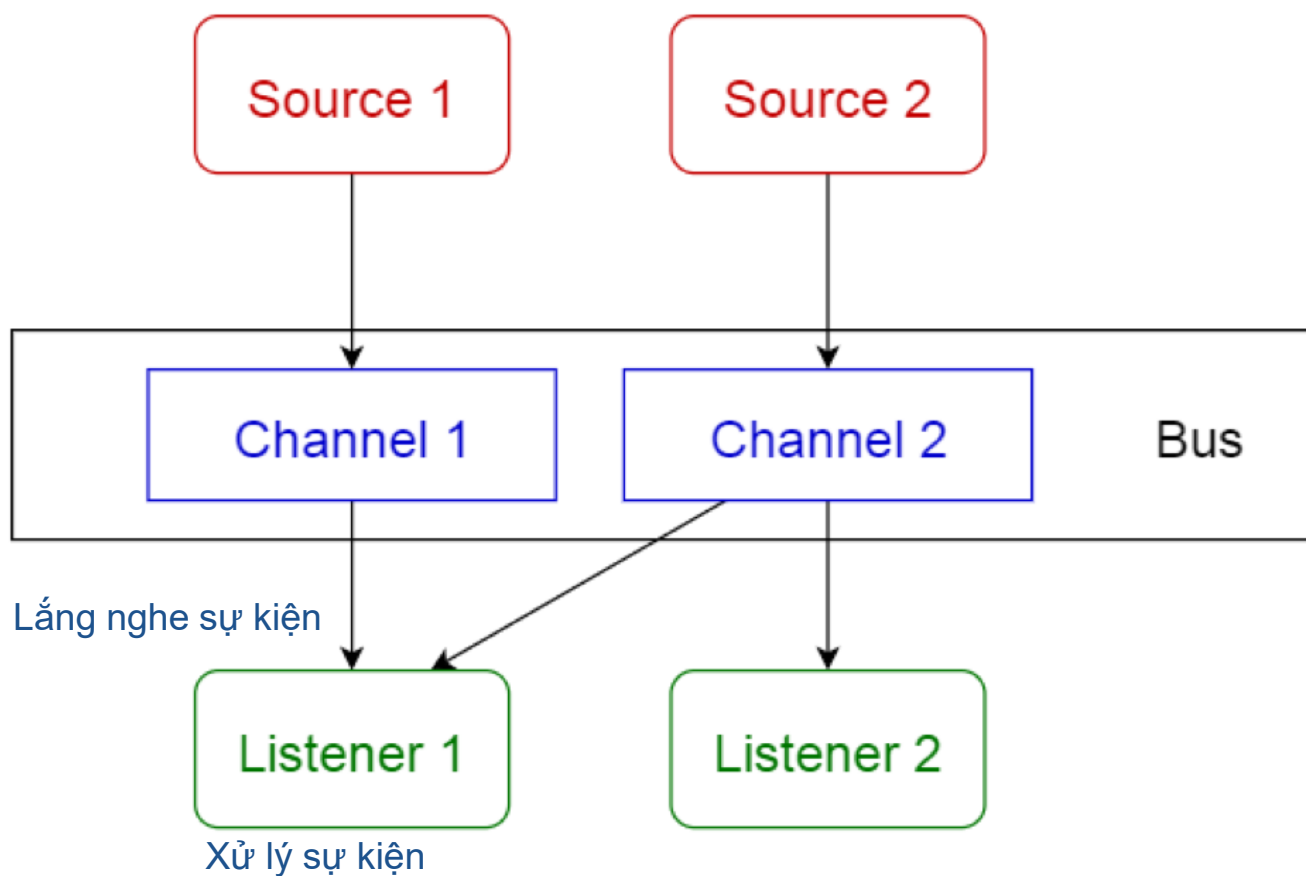
- Chủ - thợ





Mô hình kiến trúc phần mềm

- Tuyến sự kiện





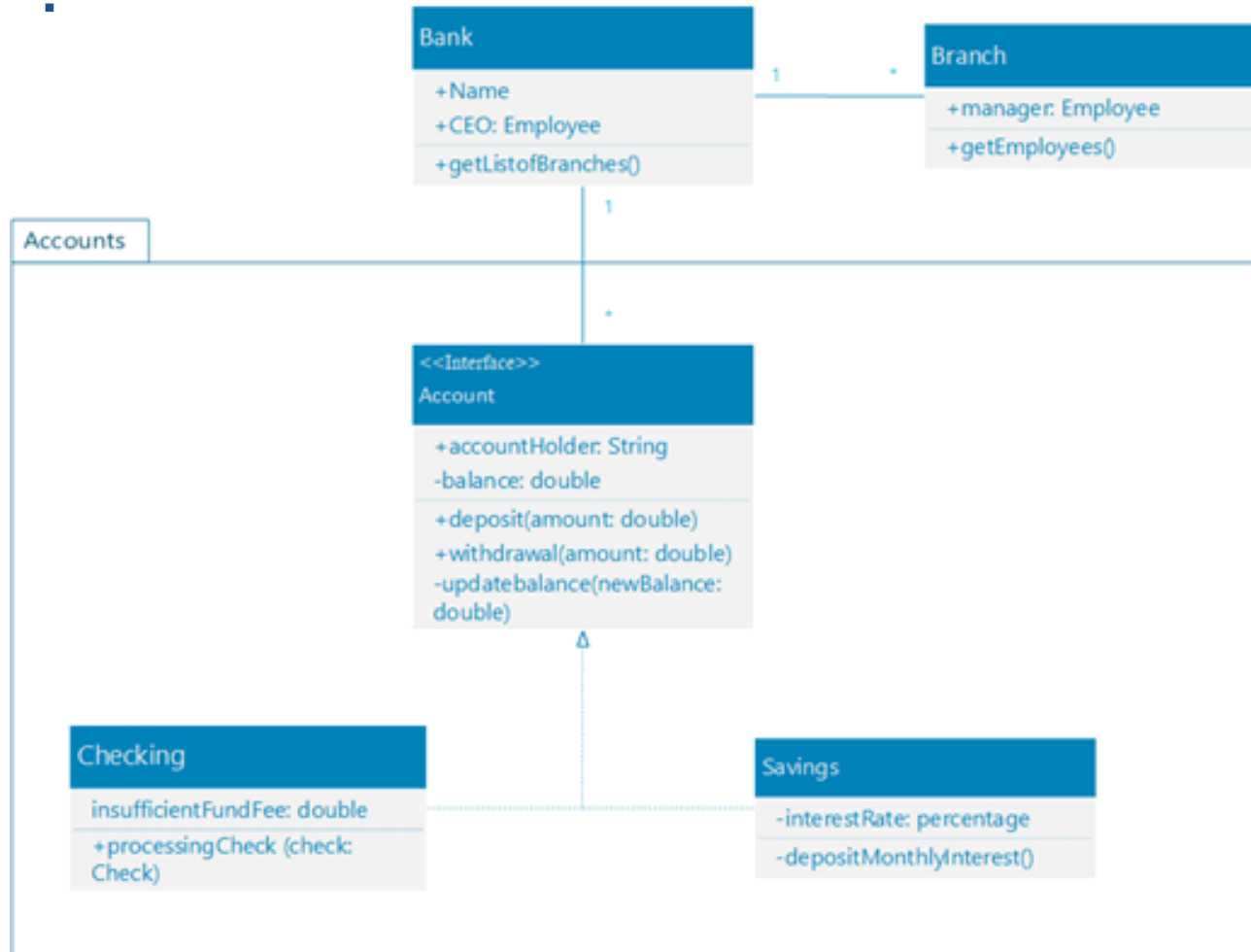
Một số loại biểu đồ

- Biểu đồ lớp:
 - Sử dụng biểu đồ lớp để tạo mô hình chung về cấu trúc của một ứng dụng.
 - Xác định:
 - các lớp của hệ thống,
 - các thuộc tính và phương thức của lớp
 - mối quan hệ giữa các đối tượng.



Biểu đồ lớp

- Ví dụ:





Biểu đồ đối tượng

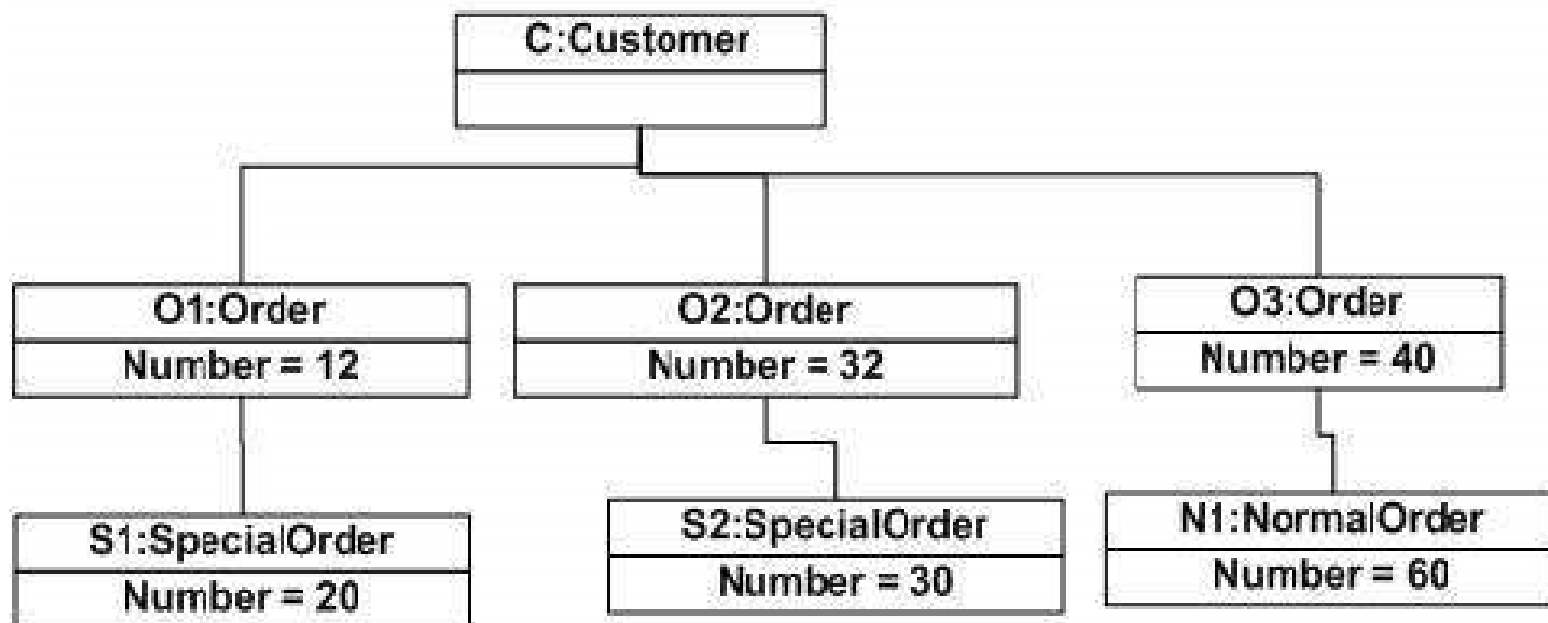
- Có thể xem là một trường hợp đặc biệt của biểu đồ lớp. Dùng để kiểm tra độ chính xác của biểu đồ lớp.
- Nó mô tả cấu trúc tĩnh của một hệ thống tại một thời điểm nhất định.
- Biểu đồ đối tượng được sử dụng để hiển thị một tập các phần tử của biểu đồ lớp nhằm nhấn mạnh mối quan hệ giữa các thể hiện của lớp ở một vài thời điểm.



Biểu đồ đối tượng

- Ví dụ

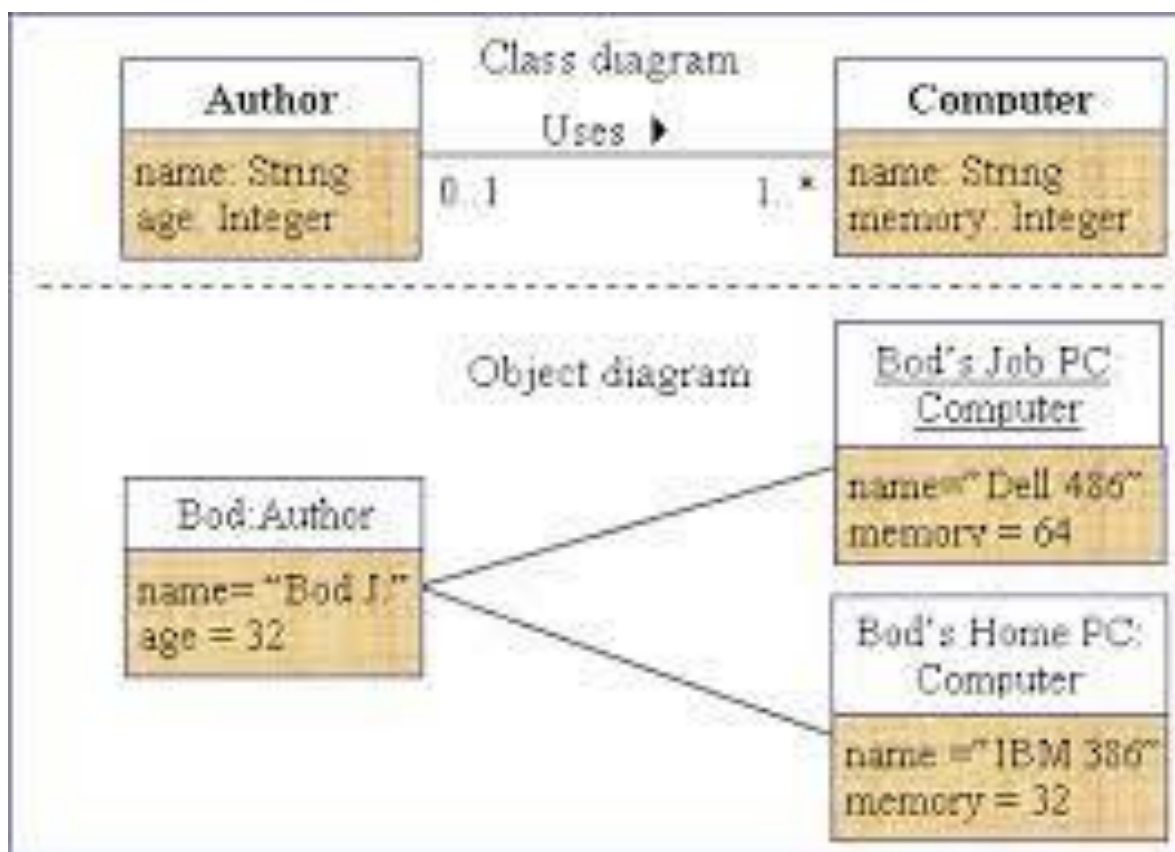
Object diagram of an order management system





Biểu đồ đối tượng

- Phân biệt biểu đồ đối tượng và biểu đồ lớp





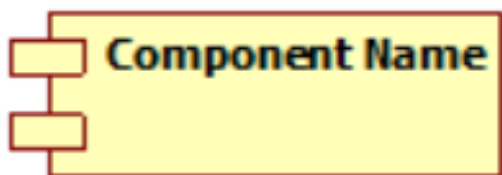
Biểu đồ đối tượng

- Biểu đồ đối tượng được sử dụng để:
 - Làm nguyên mẫu của một hệ thống
 - Mô hình hóa cấu trúc dữ liệu phức tạp
 - Hiểu hệ thống từ góc độ thực tế.



Biểu đồ thành phần

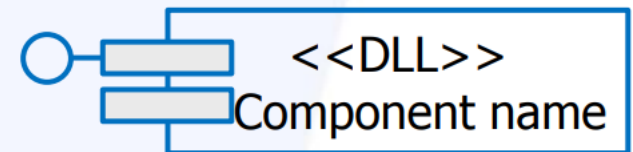
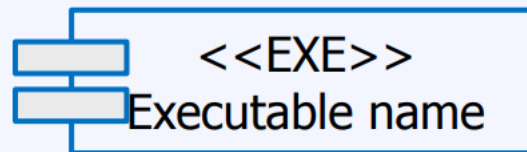
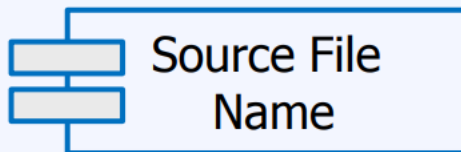
- Biểu đồ thành phần là bản vẽ cho biết cấu trúc của hệ thống theo thành phần phần mềm.
- Thành phần phần mềm được đóng gói độc lập, nó có thể được triển khai độc lập trên hệ thống và có khả năng tương tác với các thành phần khác khi thực hiện các chức năng của hệ thống





Biểu đồ thành phần

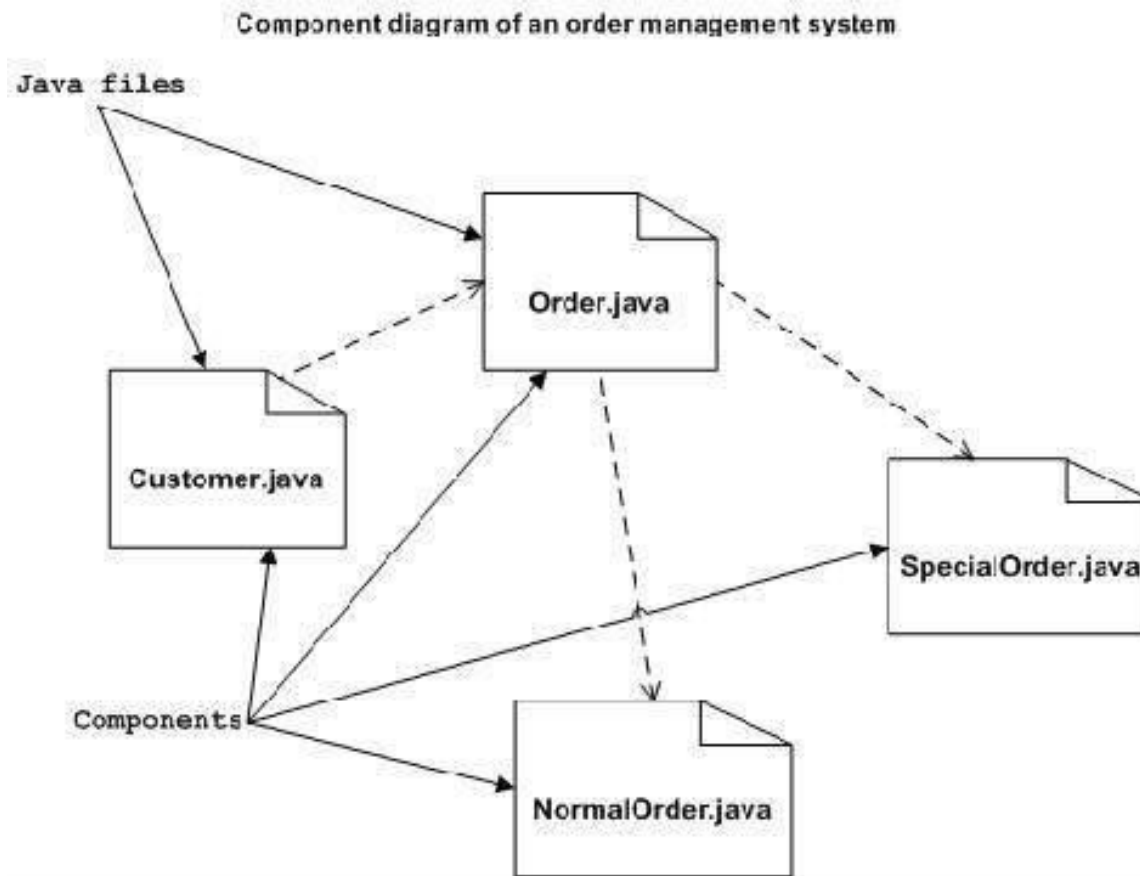
- Thành phần phần mềm giữ một chức năng rõ ràng trong hệ thống và có khả năng thay thế được.
- Một component có thể là:
 - Một source code component
 - Một runtime component
 - Một executable component





Biểu đồ thành phần

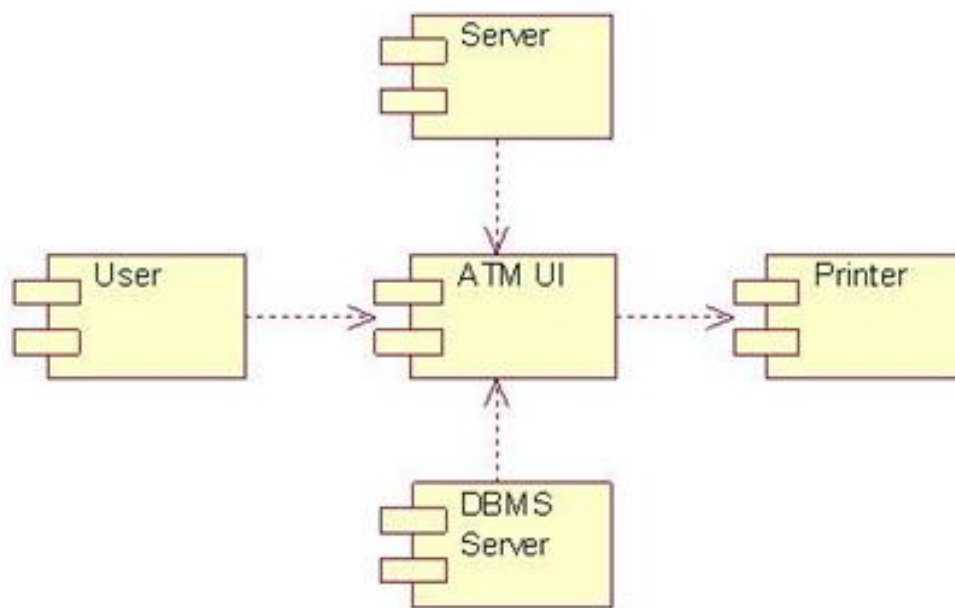
- Ví dụ 1:





Biểu đồ thành phần

- Ví dụ 2:



Component diagram cho ứng dụng máy ATM



Biểu đồ triển khai

- Biểu đồ triển khai giúp chúng ta xác định:
 - Sẽ triển khai hệ thống như thế nào?
 - Đặt các thành phần phần mềm lên hệ thống ra sao?
- Biểu diễn quan hệ vật lý giữa thành phần phần cứng và phần mềm như khi hệ thống đang thực hiện.
 - Biểu đồ triển khai có thể được sử dụng để chỉ ra thành phần nào có thể chạy trong node nào.



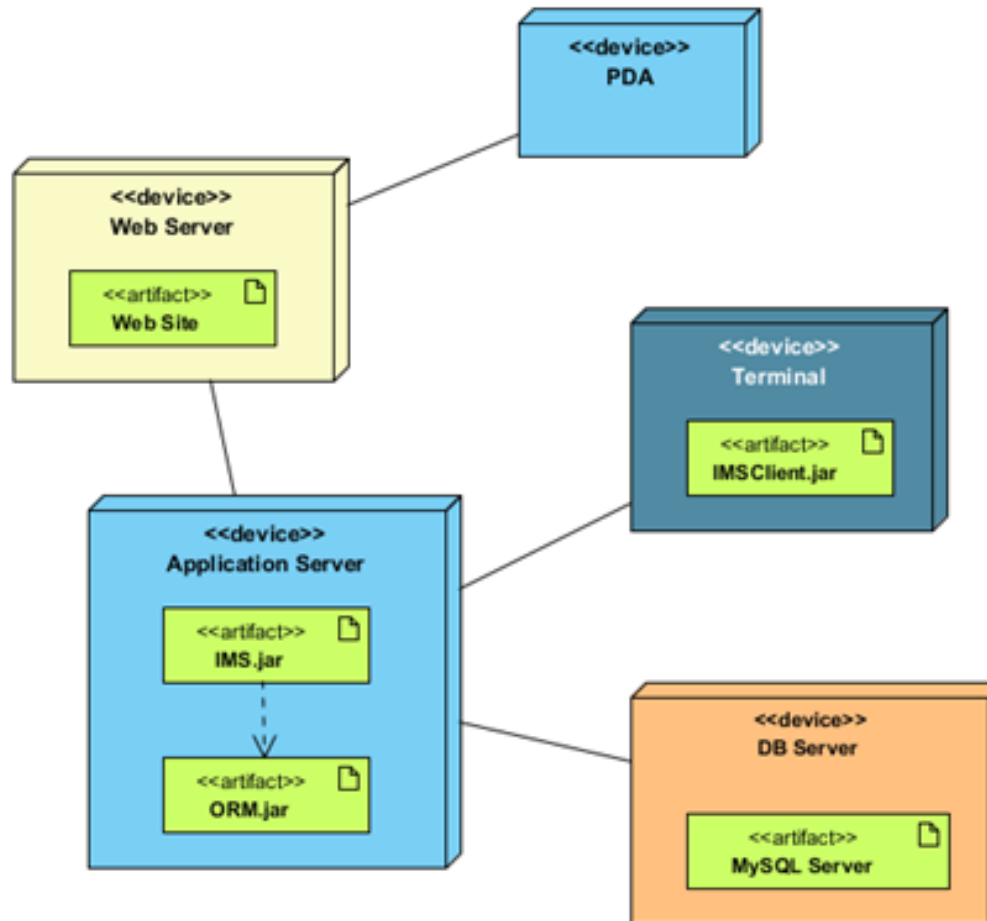
Biểu đồ triển khai

- Biểu đồ triển khai và biểu đồ thành phần có quan hệ gần nhau.
 - Biểu đồ thành phần để mô tả các thành phần, còn biểu đồ triển khai chỉ ra cách chúng được triển khai trên phần cứng.
- Mục đích của biểu đồ triển khai:
 - Hình dung ra cấu trúc liên kết của phần cứng
 - Mô tả thành phần phần cứng được sử dụng để triển khai các thành phần phần mềm.
 - Mô tả các node trong quá trình runtime.



Biểu đồ triển khai

- Ví dụ:





Biểu đồ triển khai

- Biểu đồ triển khai được sử dụng bởi các kỹ thuật viên hệ thống. Nó hiệu quả khi dùng để:
 - Thực thi hệ thống
 - Đánh giá thực thi hiệu năng của hệ thống, thực hiện khả năng chịu tải
 - Xem xét khả năng mở rộng của phần mềm
 - Dùng để bảo trì hệ thống
- Trước khi vẽ biểu đồ triển khai cần xác định
 - Các node (là một thành phần vật lý)
 - Các mối liên hệ giữa các node



Biểu đồ triển khai

- Trong đó các node là một thành phần vật lý, nó có thể là thiết bị phần cứng hoặc một môi trường nào đó mà các thành phần phần mềm được thực hiện.

Ví dụ như:

- Monitor
- Modem
- Caching Server
- Server

Phương pháp xây dựng biểu đồ lớp

- Biểu đồ lớp là biểu đồ quan trọng nhất trong thiết kế cấu trúc tĩnh của phần mềm.
- Các thành phần trong bản vẽ:
 - Các class

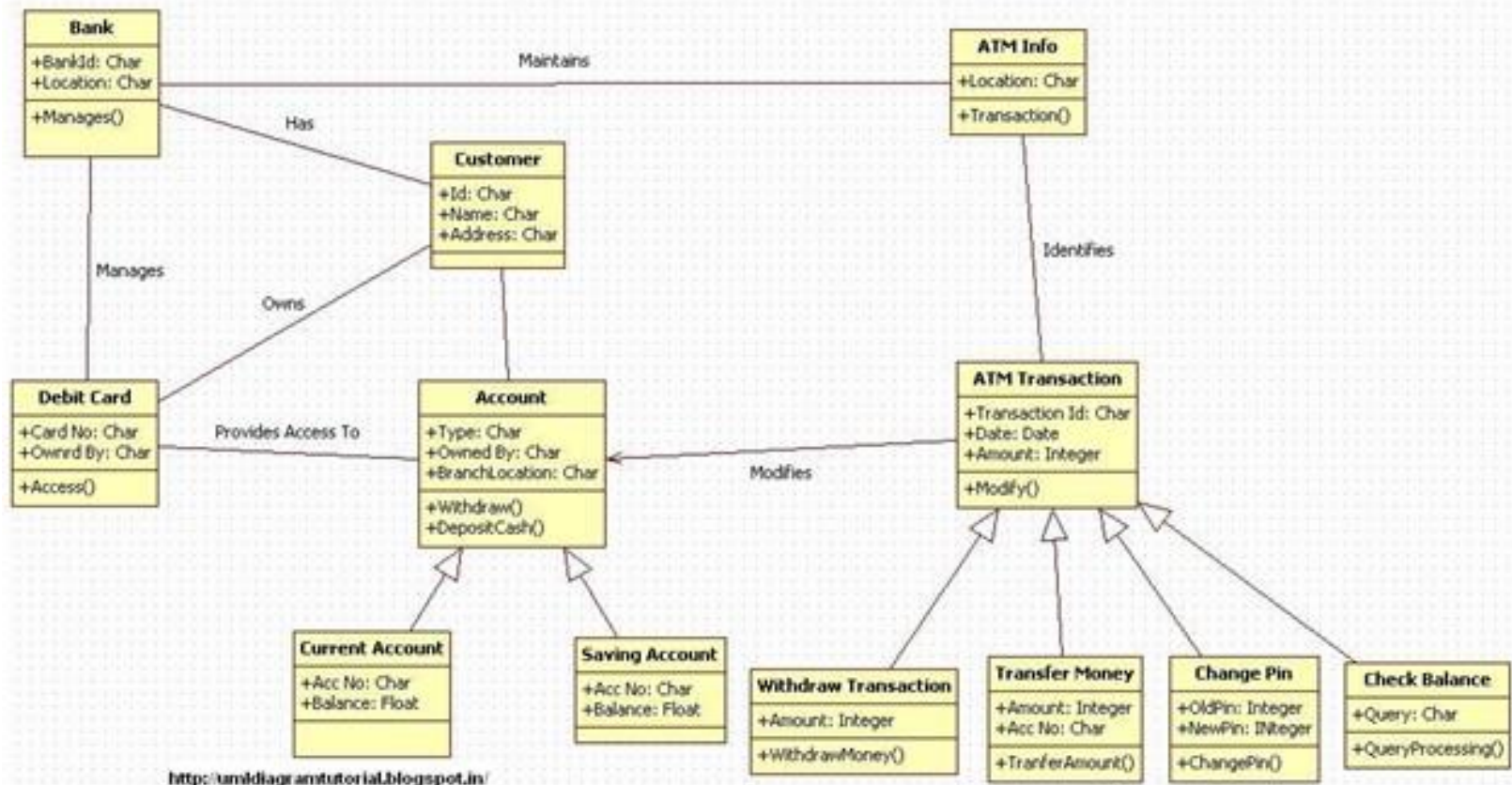
Class Name
Attributes
Methods

- Ví dụ

Customer
+CustomerCode -CustomerName -Address -BirthDay
+GetCustomerName(int CCode) +GetCustomerInfo(int CCode)

Phương pháp xây dựng biểu đồ lớp

- Các thành phần trong bản vẽ:



Ví dụ: Biểu đồ lớp của cây ATM

Phương pháp xây dựng biểu đồ lớp

– Các quan hệ (relationship):

- Thể hiện mối quan hệ giữa các class với nhau
- Trong UML, một mối quan hệ tổng quát được biểu diễn bằng một liên kết.
- Các liên kết có thể được chú thích với thông tin miêu tả về liên kết đó
- Một số mối quan hệ phổ biến:
 - Mối kết hợp (Association)-Bản số và chiều
 - Aggregation(kết tập): toàn thể và bộ phận
 - Composition(cấu thành)
 - Dependency(phụ thuộc)
 - Generalization(tổng quát hóa)- đơn/đa kế thừa
 - Realization(hiện thực hóa)

Phương pháp xây dựng biểu đồ lớp

- Các ký hiệu:

+	Public access
#	Protected access
-	Private access

- Mối quan hệ

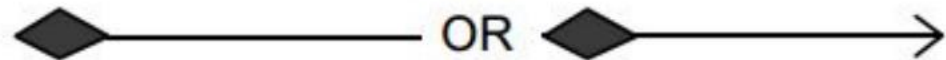
- Association



- Aggregation



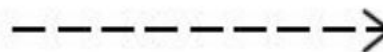
- Composition



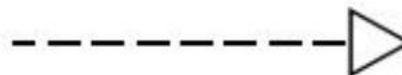
- Generalization



- Dependency

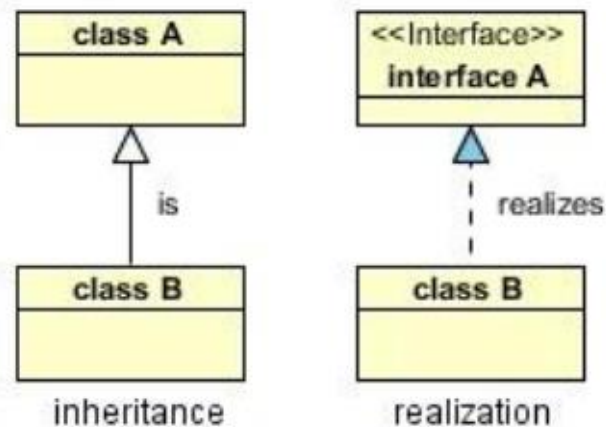
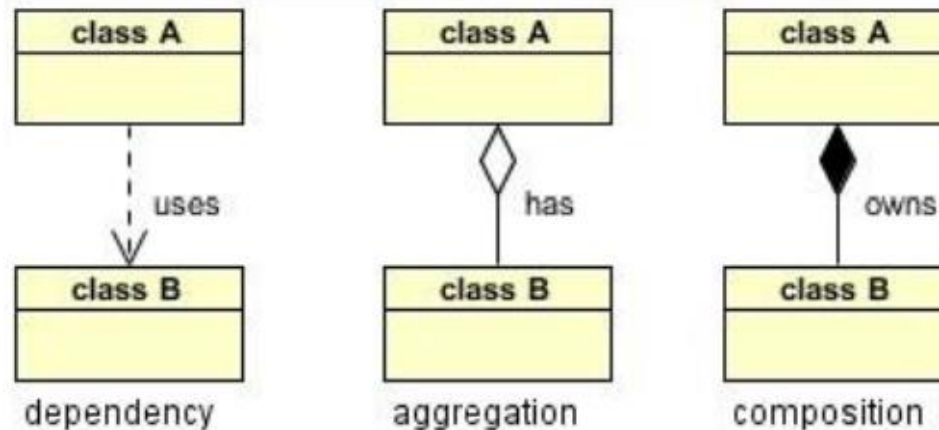


- Realization



Phương pháp xây dựng biểu đồ lớp

- Mối quan hệ



Phương pháp xây dựng biểu đồ lớp

- Phân biệt kết tập và cấu thành:
 - Kết tập mô tả mối quan hệ giữa một đối tượng lớn hơn được tạo ra từ những đối tượng nhỏ hơn.
 - Cấu thành là một loại đặc biệt của quan hệ kết tập,
 - Nó có sự liên hệ mạnh mẽ hơn về chu kỳ sống giữa lớp toàn thể và lớp bộ phận.
 - Đối tượng tổng hợp sẽ quản lý việc tạo lập và hủy bỏ những đối tượng thành phần của nó

Phương pháp xây dựng biểu đồ lớp

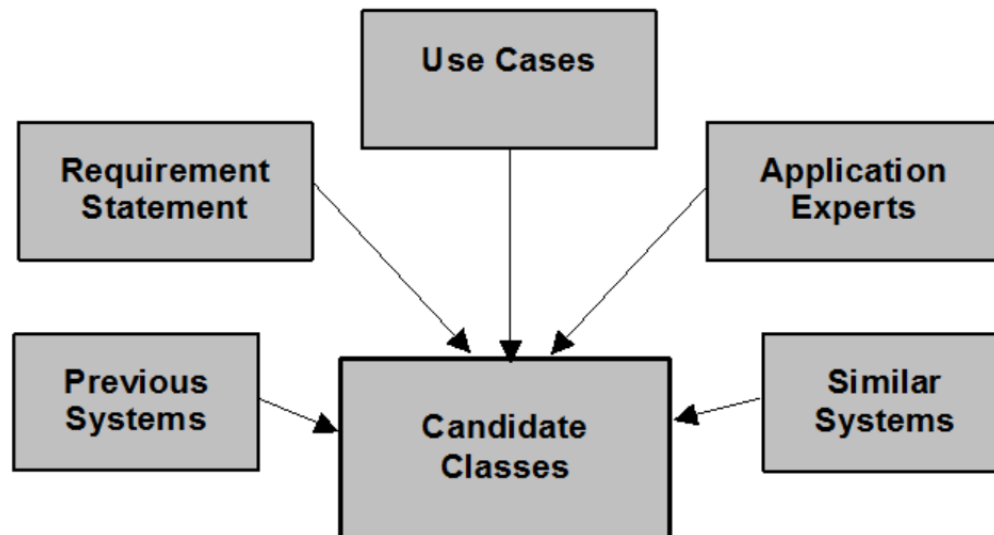
- Phân biệt kết tập và cấu thành: (tiếp)

Ví dụ:

- Aggregation: Lớp học phần và Sinh viên
 - Nếu không có sinh viên, lớp học phần không thể tồn tại.
 - Nếu không có lớp học phần, sinh viên vẫn tồn tại.
- Composition: Khoa và Sinh viên
 - Nếu không có sinh viên, khoa không thể tồn tại
 - Nếu khoa bị hủy bỏ, sinh viên không thể tồn tại.

Phương pháp xây dựng biểu đồ lớp

- Cách xây dựng bản vẽ class diagram
 - Bước 1: Tìm các class dự kiến
 - Entity class là các thực thể có thật và hoạt động trong hệ thống.
 - Các nguồn được dựa vào để xác định các class dự kiến



Phương pháp xây dựng biểu đồ lớp

- Bước 2: Tìm các thuộc tính và phương thức cho lớp
 - Tìm thuộc tính:
 - Phân tích thông tin từ các form mẫu có sẵn
 - Dựa vào hồ sơ dữ liệu
 - Tìm phương thức:
 - Phương thức là các hoạt động mà các đối tượng của lớp này có thể thực hiện.
 - Chúng ta sẽ bổ sung thêm đầy đủ cho các lớp khi phân tích biểu đồ tuần tự sau này.

Phương pháp xây dựng biểu đồ lớp

- Bước 3: Xây dựng mối quan hệ giữa các lớp và phát hiện các lớp phát sinh
 - Phân tích quan hệ giữa các lớp
 - Định nghĩa các lớp phát sinh do các quan hệ sinh ra.



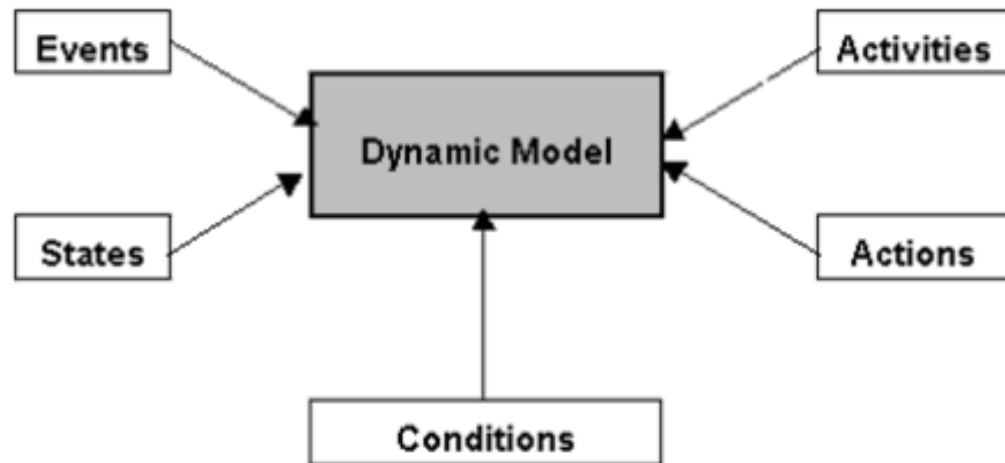
Sử dụng các loại biểu đồ mô tả tương tác của phần mềm

- Dynamic model: mô tả các hoạt động bên trong hệ thống. Bao gồm một số các bản vẽ như biểu đồ máy trạng thái, biểu đồ tuần tự, biểu đồ ca sử dụng ...
- Bài học chỉ đề cập đến interaction diagram bao gồm:
 - Biểu đồ tuần tự (Sequence diagram)
 - Biểu đồ cộng tác (Collaboration diagram)



Dynamic model

- Các thành phần cơ bản của Dynamic model:
 - Event
 - Activity
 - State
 - Action
 - Condition





Interaction Diagram

- Mục đích của hai loại biểu đồ này tương tự nhau.
 - Tập trung vào sự kiện, nhằm chỉ ra tương tác hành vi của hệ thống.
 - Mô tả luồng thông điệp trong hệ thống
 - Mô tả cấu trúc tổ chức của các đối tượng
 - Mô tả tương tác giữa các đối tượng



Biểu đồ tuần tự

- Là bản vẽ mô tả sự tương tác của các đối tượng để tạo nên chức năng của hệ thống. Bản vẽ này mô tả:
 - Chi tiết các thông điệp (message) được gửi và nhận giữa các đối tượng.
 - Chú trọng đến sự tương tác theo trình tự thời gian gửi và nhận.



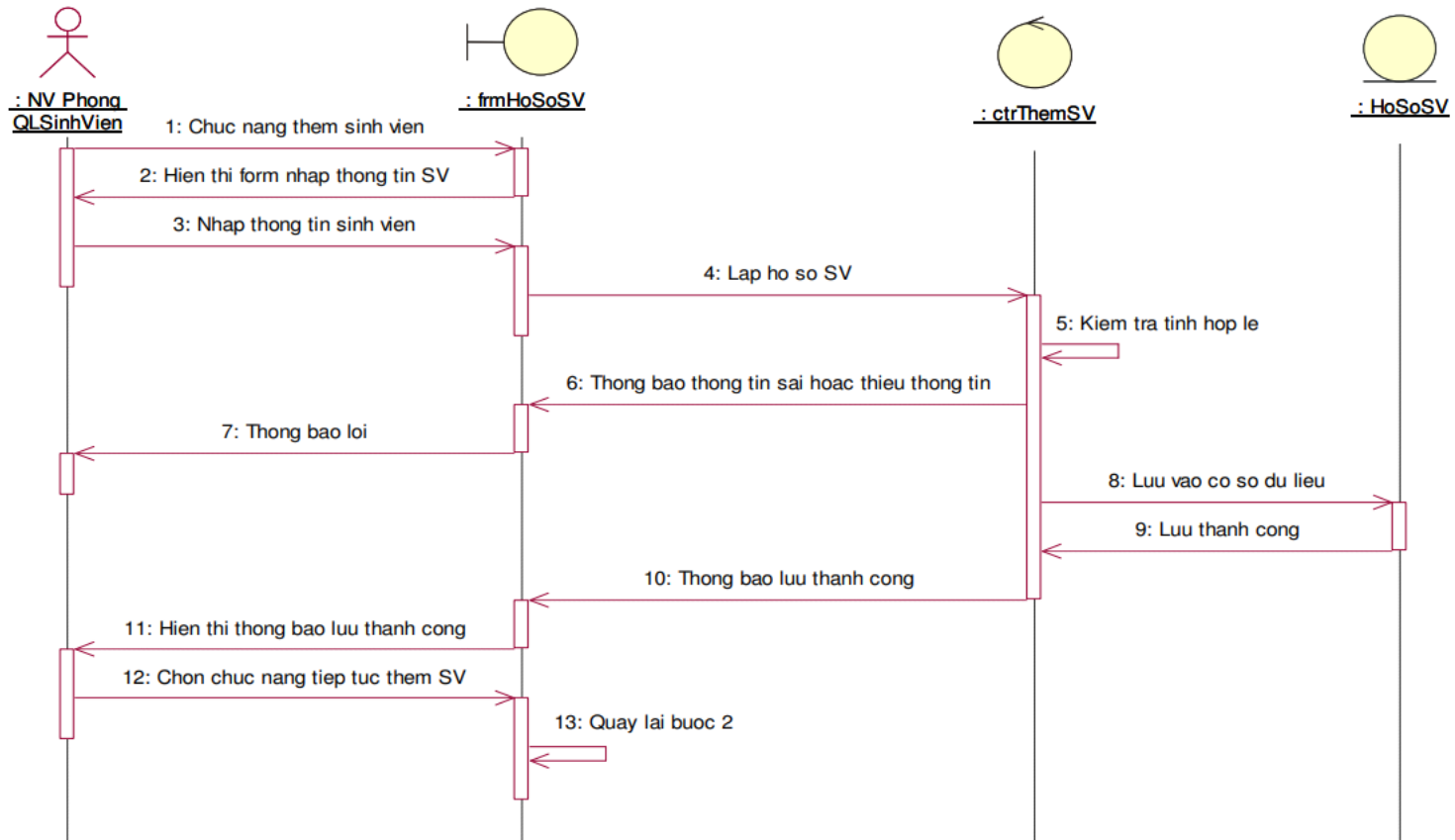
Biểu đồ tuần tự

- Cần vẽ biểu đồ tuần tự khi nào?
 - Khi cần biểu diễn chi tiết cho một UML use case.
 - Khi cần vẽ mô hình logic cho một thủ tục, một hàm hay một thao tác phức tạp.
 - Khi cần chỉ ra cách các đối tượng và các thành phần tương tác với nhau để hoàn thành một quy trình xử lý.
 - Khi cần lên kế hoạch và hiểu tính năng chi tiết của một kịch bản đang tồn tại hoặc trong tương lai.



Biểu đồ tuần tự

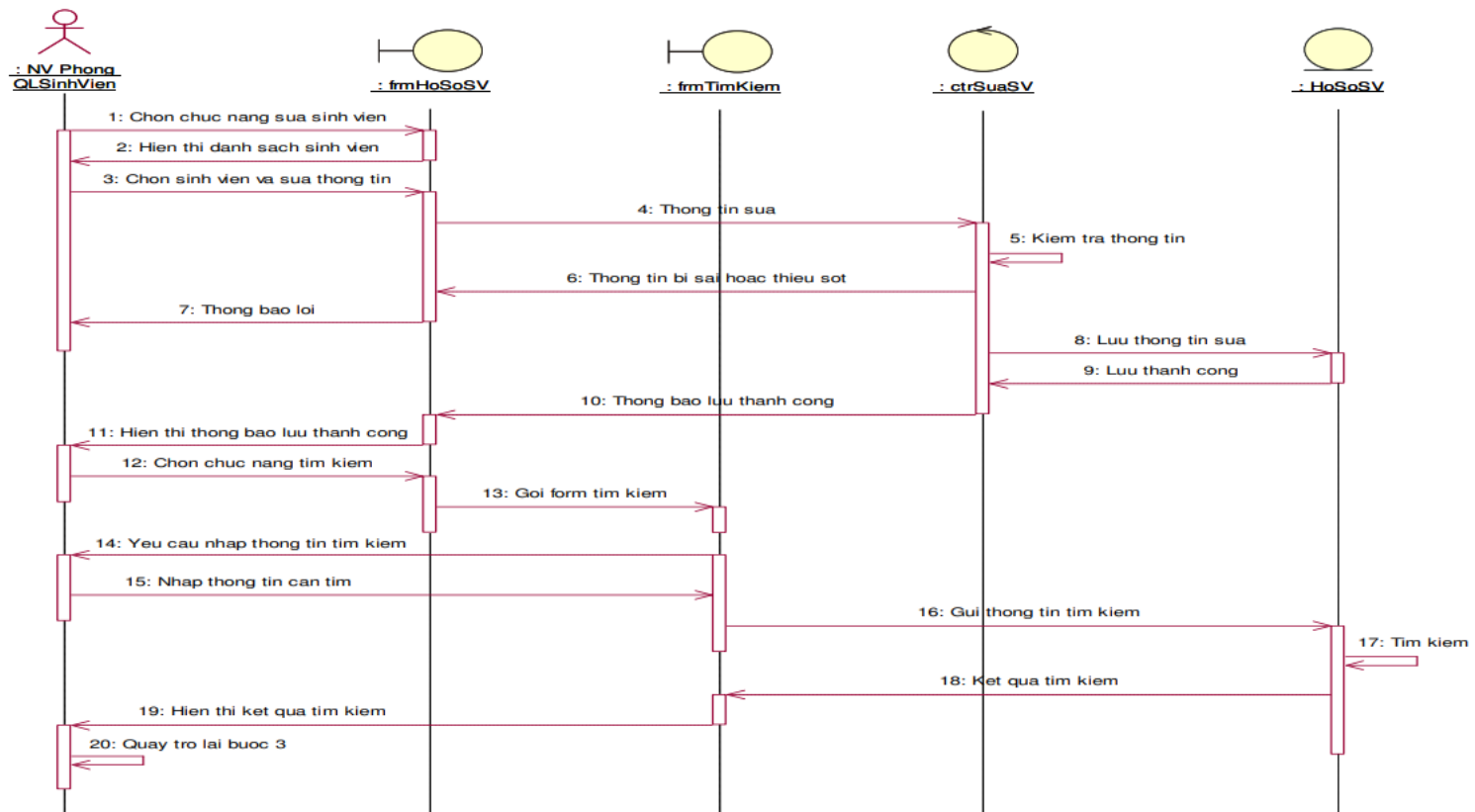
- Ví dụ: Thêm hồ sơ sinh viên





Biểu đồ tuần tự

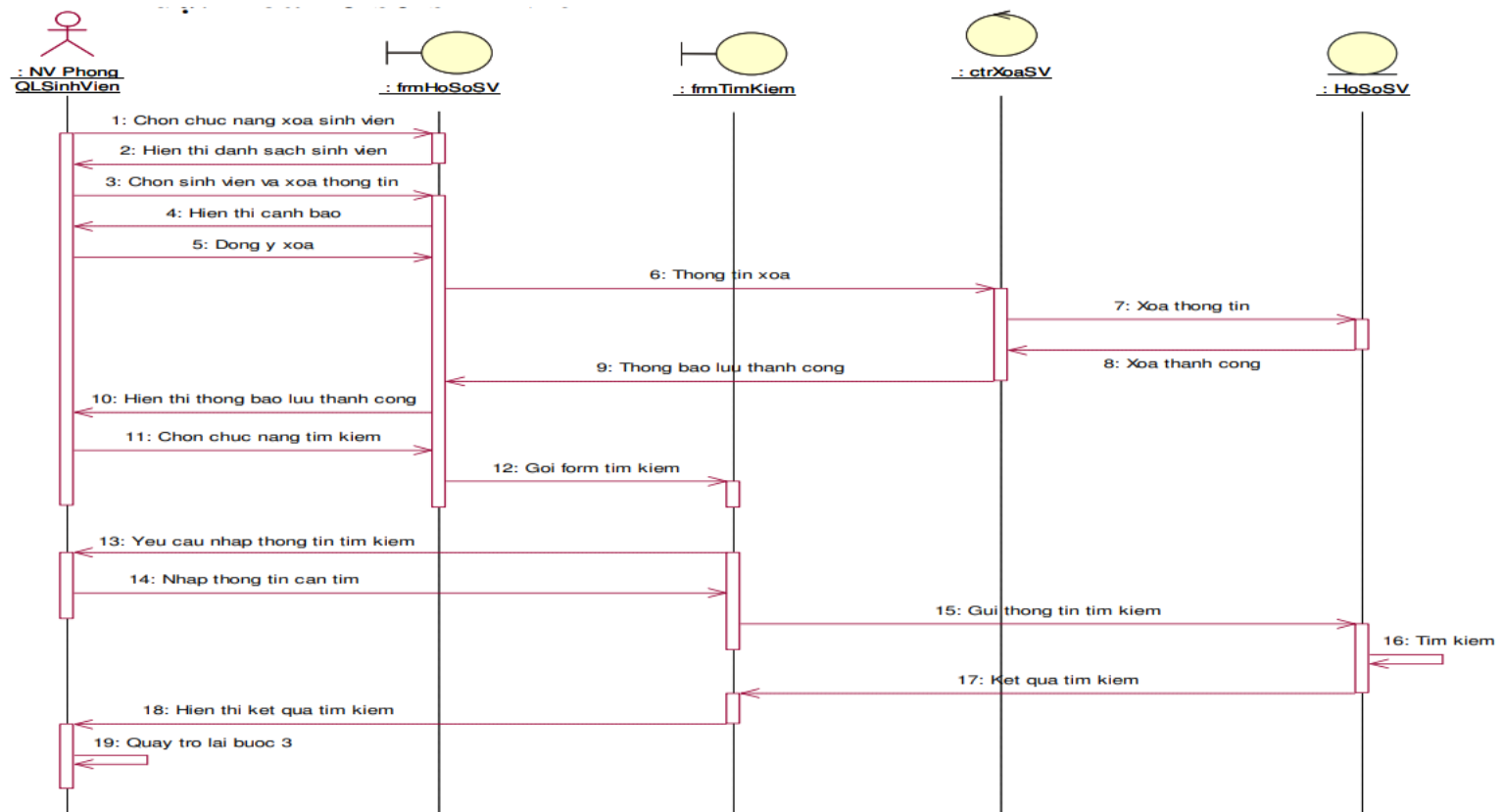
- Ví dụ: Sửa hồ sơ sinh viên





Biểu đồ tuần tự

- Ví dụ: Xóa hồ sơ sinh viên





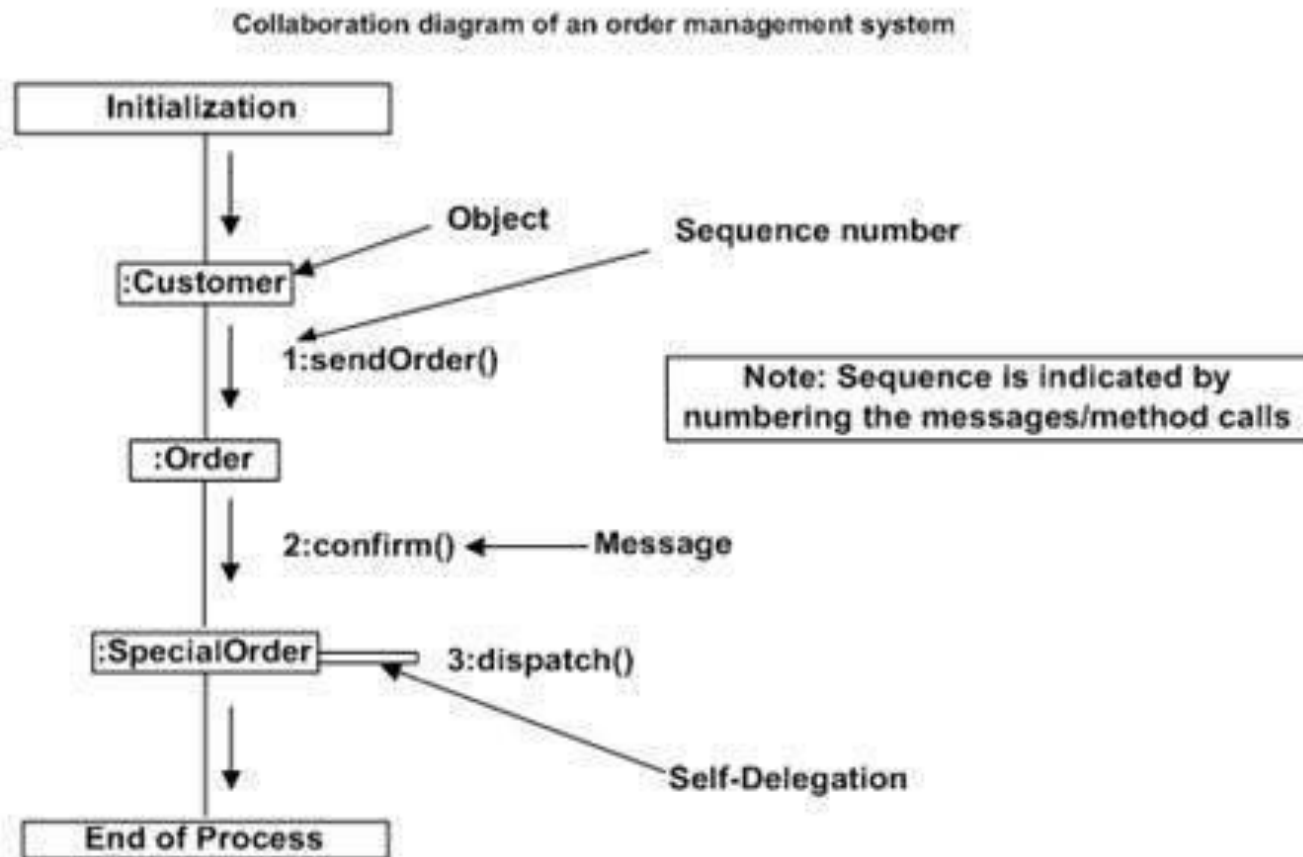
Biểu đồ cộng tác

- Tương tự như biểu đồ trình tự, biểu đồ cộng tác cũng chỉ ra luồng thực hiện nghiệp vụ cho từng chức năng của hệ thống.
- Biểu đồ cộng tác tập trung vào:
 - Quan hệ giữa các đối tượng
 - Cấu trúc tổ chức của các đối tượng
 - Hành vi của các đối tượng (điều mà sơ đồ trình tự thường tránh né)



Biểu đồ cộng tác

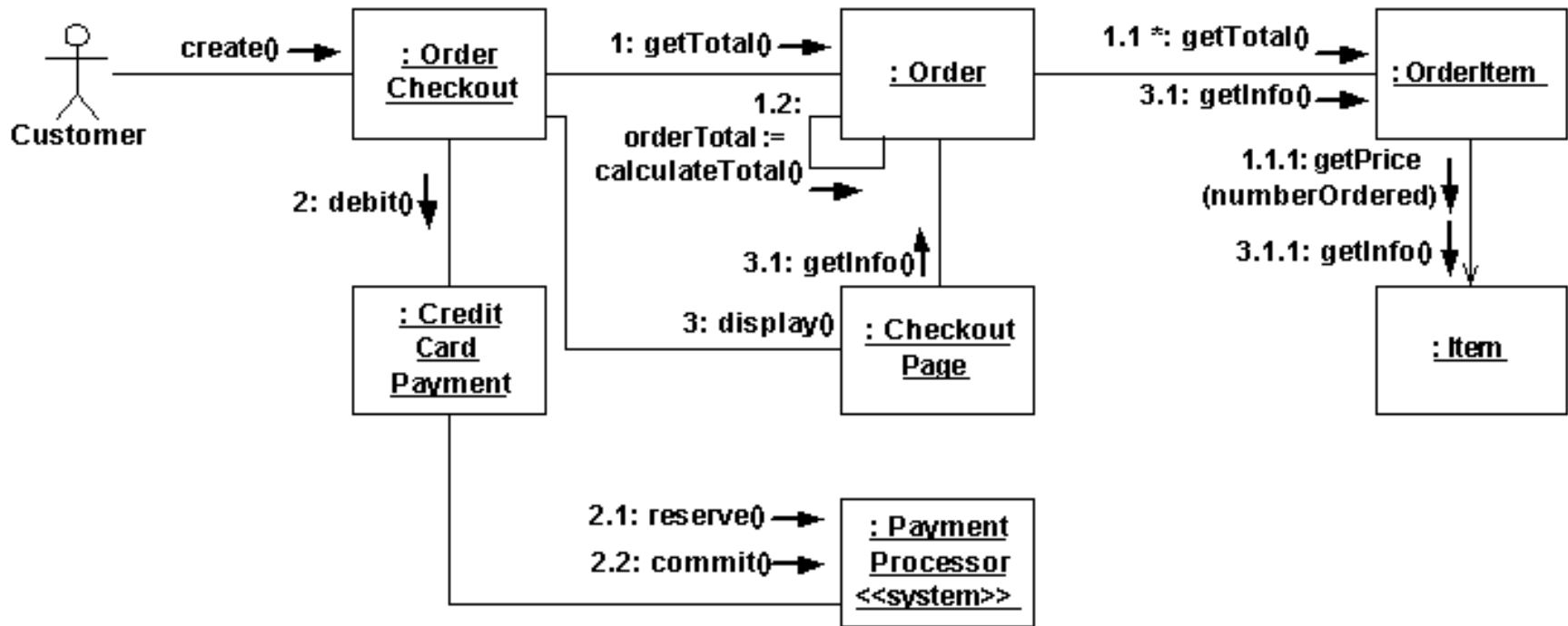
- Ví dụ: hệ thống quản lý chọn món





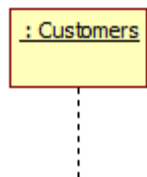
Biểu đồ cộng tác

- Ví dụ: Hệ thống quản lý đặt hàng

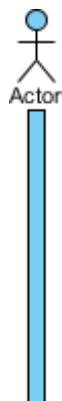


Phương pháp xây dựng biểu đồ Sequence

- Các thành phần của biểu đồ sequence:
 - Object
 - Message



- Actor



- Trục tọa độ:

Trục ngang thể hiện các đối tượng

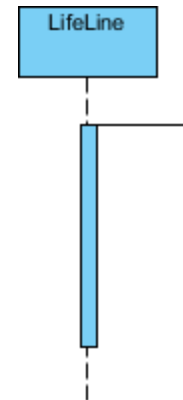
Trục đứng thể hiện thời gian

Phương pháp xây dựng biểu đồ Sequence

- Các thành phần (tiếp)
 - Đường đời



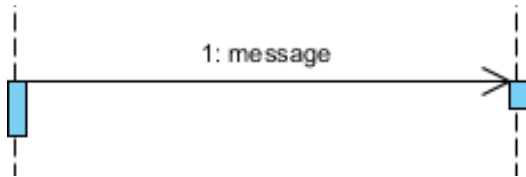
- Activation
(thời gian hoạt động)



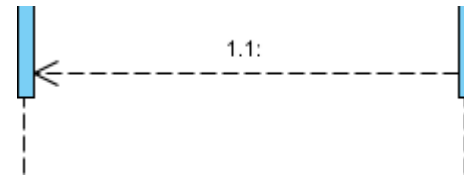
Phương pháp xây dựng biểu đồ Sequence

- Các loại thông điệp (message)

Call message



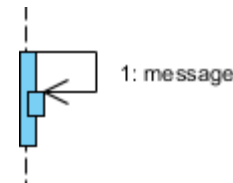
Return message



Self message



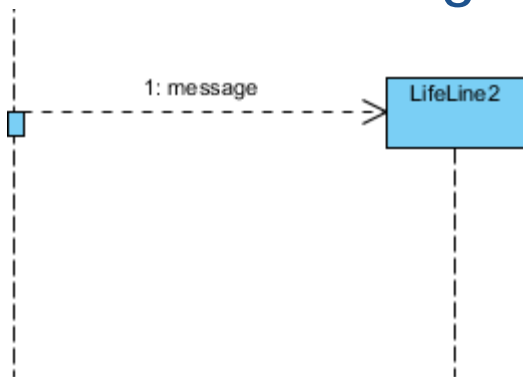
Recursive message



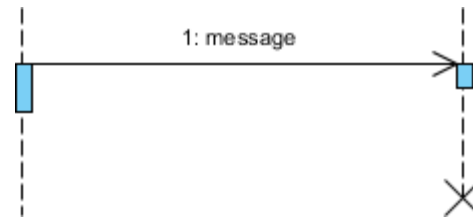
Phương pháp xây dựng biểu đồ Sequence

- Các loại thông điệp (tiếp)

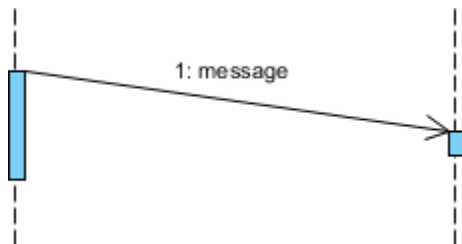
Create message



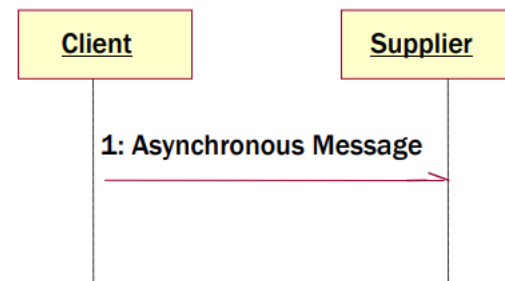
Destroy message



Duration message



Asynchronous message

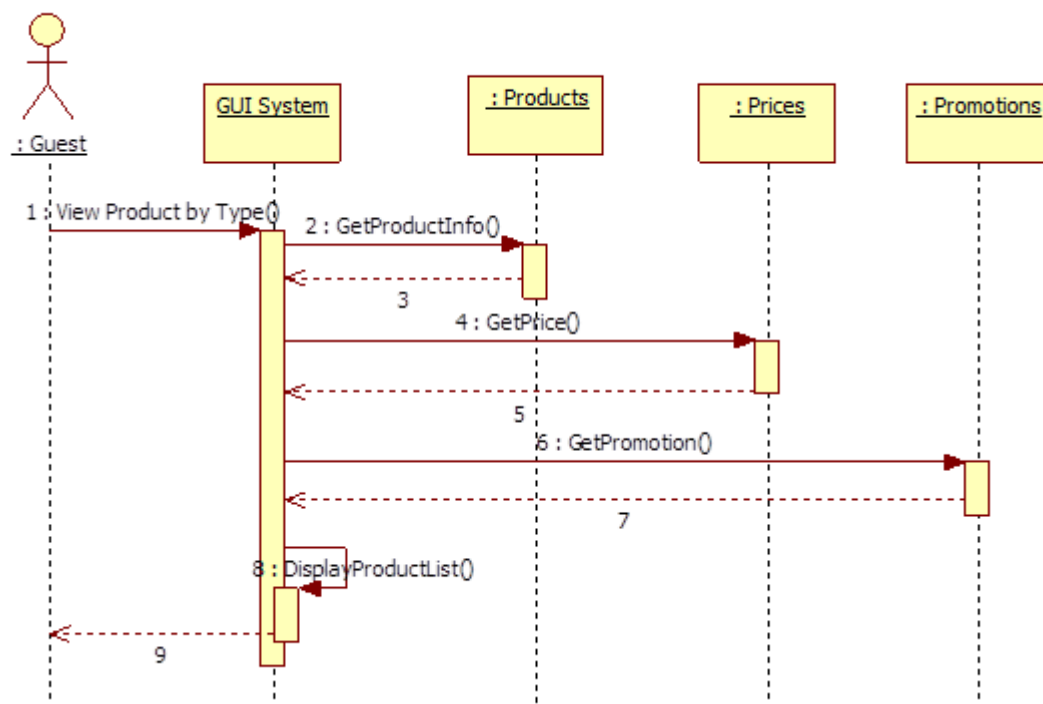


Phương pháp xây dựng biểu đồ Sequence

- Các bước xây dựng biểu đồ tuần tự:
 - Bước 1: Xác định chức năng cần thiết kế
 - Bước 2: Dựa vào Activity Diagram để xác định các bước thực hiện theo nghiệp vụ
 - Bước 3: Đối chiếu với Class Diagram để xác định lớp trong hệ thống tham gia vào nghiệp vụ
 - Bước 4: Vẽ biểu đồ tuần tự (sequence diagram)
 - Bước 5: Cập nhật lại bản vẽ Class Diagram

Phương pháp xây dựng biểu đồ Sequence

- Ví dụ: Chức năng xem sản phẩm theo chủng loại





Nội dung

→ 3. Các nguyên tắc thiết kế

- Thừa kế và kết hợp thành phần
- Các lỗi thiết kế thường gặp
- Tái cấu trúc thiết kế
- Nguyên tắc thiết kế Agile

4. Các mẫu thiết kế

5. Thiết kế giao diện người sử dụng

6. Thiết kế dữ liệu



Các nguyên tắc thiết kế

- Thiết kế là giai đoạn quan trọng quyết định chất lượng phần mềm. Do vậy cần đúc kết tuân thủ các nguyên tắc trong thiết kế để đảm bảo sản phẩm mã code sau cùng phải thỏa mãn các tính chất:
 - Bảo trì
 - Khả năng mở rộng
 - Tính modul



Các nguyên tắc thiết kế

- Có nhiều nguyên tắc thiết kế khác nhau áp dụng cho từng chiến lược thiết kế. Ví dụ:
 - Thiết kế hướng đối tượng nên tuân theo
 - Các nguyên tắc SOLID của Robert C. Martin và Michael Feathers
 - Các nguyên tắc GRASP của Craig Larman
 - Một số nguyên tắc khác như độ kết dính cao, DRY, KISS, YAGNI...



Các nguyên tắc thiết kế

- Các tính chất khiến hướng đối tượng trở nên hiệu quả chính là tuân thủ: Tính đóng gói, tính trừu tượng, tính kế thừa và tính đa hình.
 - Cần phối hợp các tính chất này với nhau để tăng hiệu quả của ứng dụng.
 - Các nguyên tắc nêu trên là sự phối hợp giữa các tính chất này.
 - Trong nội dung đề cương chỉ đề cập đến thừa kế và kết hợp thành phần.



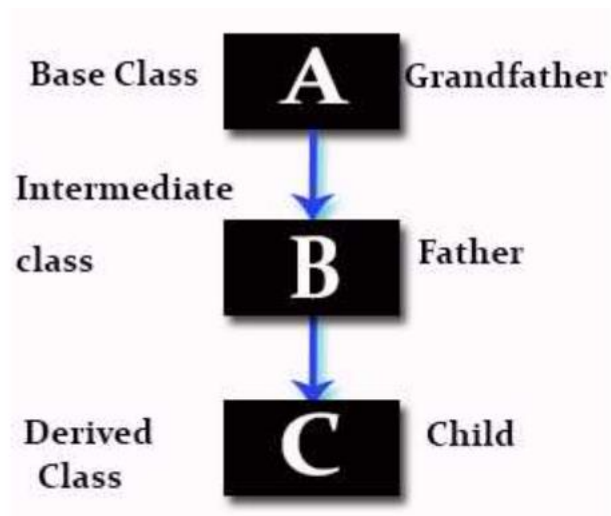
Thừa kế và kết hợp thành phần

- Trong lập trình hướng đối tượng, để tái sử dụng mã code người ta thường sử dụng cách thiết lập mối quan hệ giữa các lớp.
 - Có hai cách cơ bản để thiết lập các mối quan hệ này đó là kế thừa và kết hợp thành phần.
 - Lớp mới là lớp tái sử dụng mã của lớp cũ. Lớp mới này có thể kế thừa từ lớp cũ hoặc nhúng mã từ một hoặc nhiều lớp cũ (kết hợp thành phần)



Thừa kế và kết hợp thành phần

- Kế thừa:
 - Là phương pháp mà theo đó một lớp có thể có được các đặc điểm(phương thức và thuộc tính) của một hoặc nhiều lớp khác.
 - Lớp hiện có (lớp cũ) có quan hệ cha-con, ông-cháu... với lớp mới





Thừa kế và kết hợp thành phần

- Kết hợp thành phần:
 - Trong kết hợp thành phần chúng ta sử dụng đối tượng của lớp cũ (lớp hiện có) bên trong một lớp mới (lớp cần xây dựng)
 - Không giống như kế thừa được xác định trong thời gian biên dịch, thì kết hợp thành phần được xác định động trong thời gian runtime.
 - Kế thừa nếu thiết kế không tốt sẽ phá vỡ tính đóng gói còn kết hợp thành phần thì không.
 - Kết hợp thành phần là mô hình tái sử dụng mã code tốt hơn so với kế thừa lớp.



Các lỗi thiết kế thường gặp

- Các lỗi thường gặp khi thiết kế biểu đồ lớp
 - Danh sách các biến thành viên, các hàm thành viên không đầy đủ.
 - Do lỗi trong quá trình trừu tượng hóa đối tượng từ thực tế.
 - Hoặc do lỗi trong giai đoạn phân tích đặc tả yêu cầu của dự án chưa rõ ràng hoặc thiếu đầy đủ...
 - Xác định mối liên hệ giữa các lớp không chính xác.
 - Thường là hay nhầm lẫn giữa aggregation và composition.



Các lỗi thiết kế thường gặp

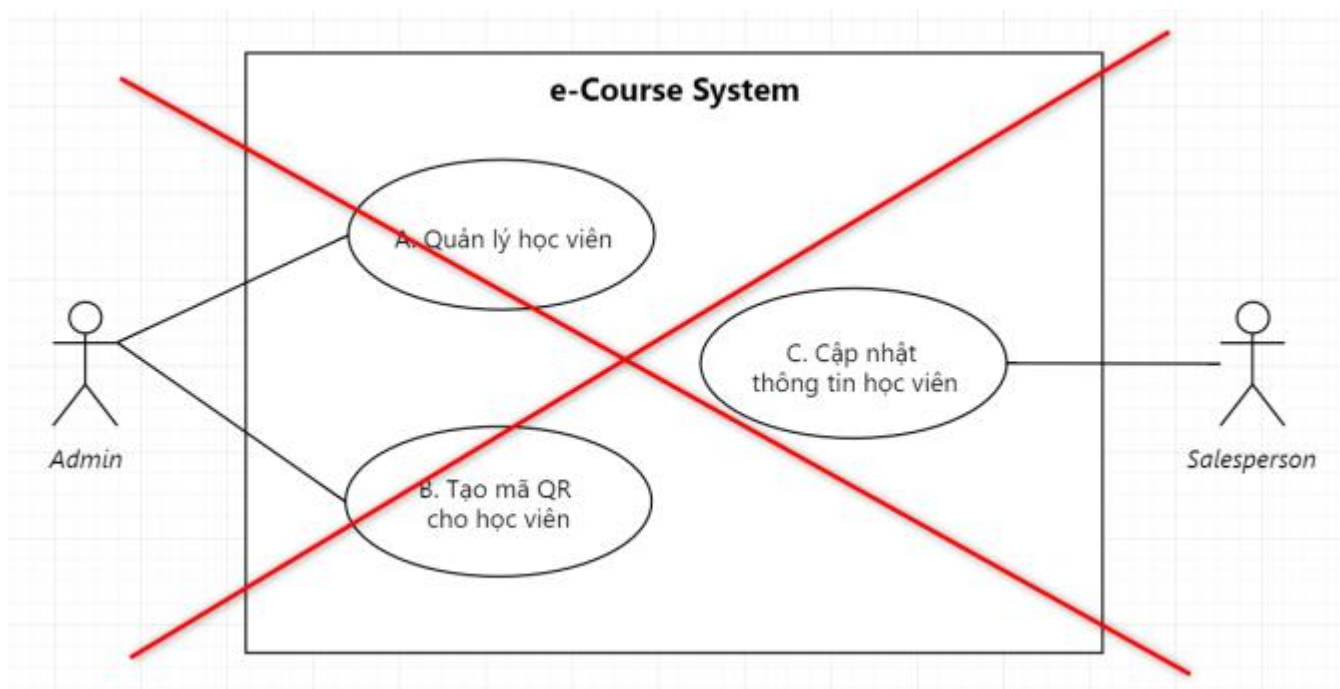
- Các lỗi thường gặp khi thiết kế biểu đồ use-case:
 - Lỗi khi đặt tên hoặc tên quá dài
 - Vẽ Use Case mà thành biểu đồ phân rã chức năng. Cần tránh:
 - Vẽ use case nhưng chưa mang được góc nhìn của end-user do nắm thiếu thông tin:
 - Tương tác giữa end-user với hệ thống
 - Tương tác giữa hệ thống với hệ thống bên ngoài
 - Vẽ use case bị lệch cấp độ requirement với nhau



Các lỗi thiết kế thường gặp

- Các lỗi thường gặp khi thiết kế biểu đồ use-case: (Tiếp)

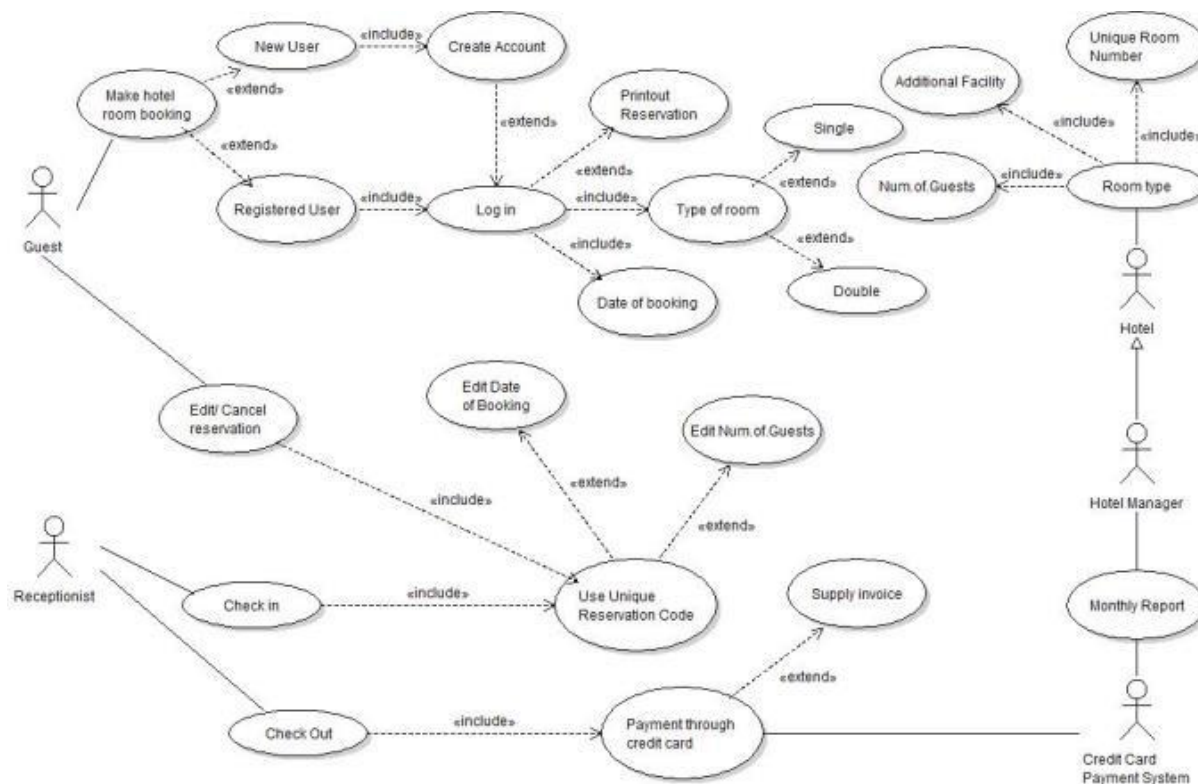
Ví dụ: Lệnh cấp độ





Các lỗi thiết kế thường gặp

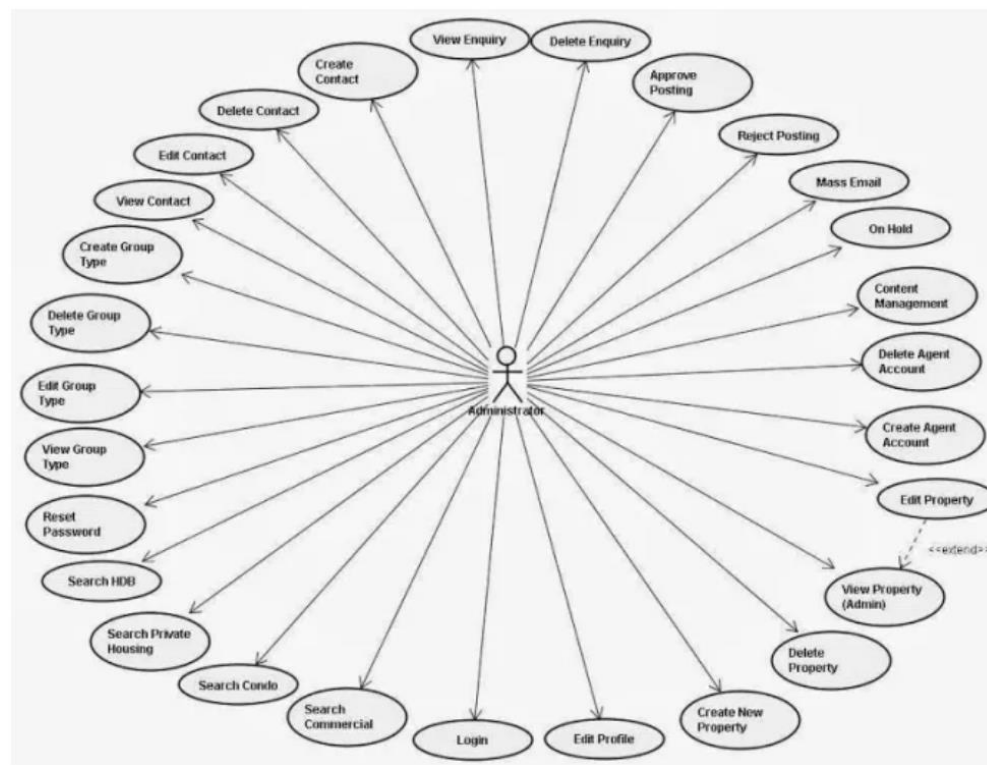
- Các lỗi thường gặp khi thiết kế biểu đồ use-case: (Tiếp)
 - Rối nùi Use Case





Các lỗi thiết kế thường gặp

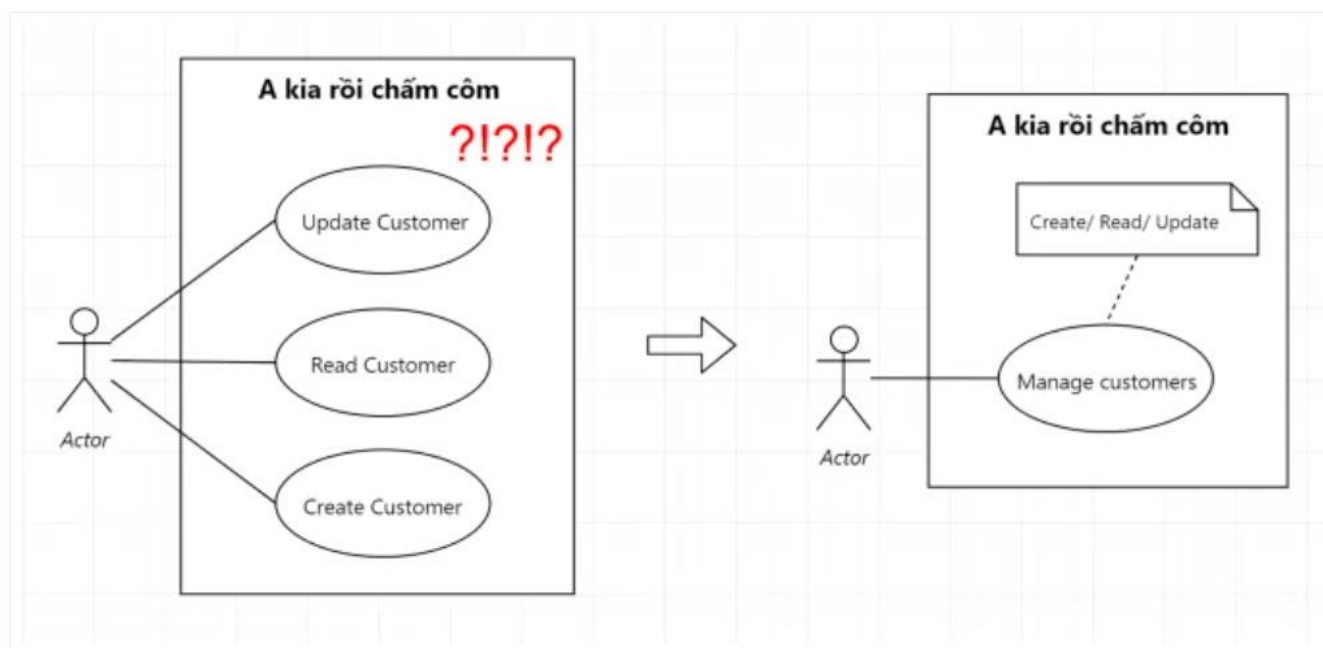
- Các lỗi thường gặp khi thiết kế biểu đồ use-case: (Tiếp)
 - Rối nùi Use Case





Các lỗi thiết kế thường gặp

- Các lỗi thường gặp khi thiết kế biểu đồ use-case: (Tiếp)
 - Quá chi tiết các chức năng CRUD



Thay vì vẽ như bên trái, thì hãy vẽ như bên phải.



Tham khảo

- Use case
 - <https://www.uml-diagrams.org/use-case-diagrams.html>
 - <https://www.smartdraw.com/use-case-diagram/>



Các lỗi thiết kế thường gặp

- Những sai lầm thường gặp trong thiết kế CSDL:
 - Bỏ qua mục đích của dữ liệu
 - Chuẩn hóa yếu
 - Ràng buộc yếu
 - Không tận dụng những tính năng của DB Engine
 - Khóa chính kết hợp
 - Chỉ mục yếu
 - Quy tắc đặt tên không tốt ...



Các lỗi thiết kế thường gặp

- Những sai lầm nên tránh khi thiết kế UI:
 - Các thành phần không được căn chỉnh
 - Diện tích vùng bấm quá nhỏ (đặc biệt trên mobile app)
 - Các nút chưa có sự phân cấp
 - Mù quáng chạy theo xu hướng (bắt trend) mà quên đi trải nghiệm người dùng (UX)
 - Hình ảnh kém thu hút



Các lỗi thiết kế thường gặp

- Những sai lầm nên tránh khi thiết kế trải nghiệm người dùng (UX):
 - Tập trung vào thiết kế giao diện hơn là giải quyết vấn đề chính (trải nghiệm người dùng)
 - Xây dựng sản phẩm mà không nghiên cứu, khảo sát người dùng.
 - Gây căng thẳng cho người dùng khiến họ nghĩ mình sai “tệ hại” và từ bỏ.
 - Thiết kế chỉ xoay quanh công nghệ thay vì xoay quanh người dùng.
 - Sản phẩm chỉ đáp ứng tối ưu nhu cầu với các người dùng mới, không đáp ứng được với người dùng thường xuyên lâu dài.
 - Để các designer phải “cô độc” khi thiết kế



Tái cấu trúc thiết kế

- Tại sao cần tái cấu trúc?
 - Phần mềm sẽ không hiệu quả nếu không theo kịp sự thay đổi của thế giới và các đối thủ cạnh tranh.
 - Khi đối thủ có thêm các tính năng mới hữu ích, nếu chúng ta không thay đổi chúng ta sẽ lạc hậu.
 - Vì vậy khi phát triển phần mềm chúng ta phải luôn thay đổi để thích nghi. Một trong những cách hiện thực hóa điều đó là tái cấu trúc



Tái cấu trúc thiết kế

- Tái cấu trúc thiết kế là:
 - Thay đổi hành vi bên trong của phần mềm để phần mềm dễ hiểu và dễ sửa đổi và dễ mở rộng hơn,
 - Mà không làm thay đổi hành vi nhìn thấy từ bên ngoài. Có nghĩa là phần mềm vẫn có cùng chức năng như trước.
 - Tái cấu trúc thiết kế giúp mã code trở nên “sạch” và hiệu quả hơn.
 - Cần chú ý phân biệt phép tối ưu hiệu năng không phải là tái cấu trúc thiết kế.



Tái cấu trúc thiết kế

- Khi nào cần tái cấu trúc:
 - Tái cấu trúc khi thêm chức năng
 - Tái cấu trúc khi sửa lỗi
 - Tái cấu trúc khi review mã code





Tái cấu trúc thiết kế

- Hai chiếc mũ của Kent Beck
 - Khi thực hành tái cấu trúc cần phối hợp giữa hai hoạt động khác nhau:
 - Thêm chức năng
 - Và tái cấu trúc
 - Khi thêm chức năng, có thể cần tái cấu trúc trước để việc thêm dễ dàng hơn. Sau khi code trở nên “sạch” để sẵn sàng hơn, chúng ta nên chuyển sang tiếp tục thêm chức năng và lặp lại sự hoán đổi hai “chiếc mũ” này cho đến khi có được hệ thống cần xây dựng.



Tái cấu trúc thiết kế

- Mục đích của việc tái cấu trúc:
 - Giúp cải thiện thiết kế của phần mềm
 - Là cách giải quyết vấn đề technical debt (nợ kỹ thuật)
 - Mã dễ đọc, dễ hiểu hơn
 - Lợi ích hệ quả:
 - Mở rộng hệ thống dễ dàng hơn
 - Mã dễ tầm lỗi hơn
 - Lập trình nhanh hơn
 - Giúp hệ thống khó bị tổn thương hơn khi được khắc phục các lỗi hổng.



Các nguyên tắc thiết kế Agile

1. Ưu tiên cao nhất của chúng tôi là thỏa mãn khách hàng thông qua việc chuyển giao sớm và liên tục các phần mềm có giá trị.
2. Chào đón việc thay đổi yêu cầu, thậm chí rất muộn trong quá trình phát triển. Các quy trình linh hoạt tận dụng sự thay đổi cho các lợi thế cạnh tranh của khách hàng.
3. Thường xuyên chuyển giao phần mềm chạy tốt tới khách hàng, từ vài tuần đến vài tháng, ưu tiên cho các khoảng thời gian ngắn hơn.



Các nguyên tắc thiết kế Agile

4. Nhà kinh doanh và nhà phát triển phải làm việc cùng nhau hàng ngày trong suốt dự án.
5. Xây dựng các dự án xung quanh những cá nhân có động lực. Cung cấp cho họ môi trường và sự hỗ trợ cần thiết, và tin tưởng họ để hoàn thành công việc.
6. Phương pháp hiệu quả nhất để truyền đạt thông tin tới nhóm phát triển và trong nội bộ nhóm phát triển là hội thoại trực tiếp.
7. Phần mềm chạy tốt là thước đo chính của tiến độ.

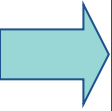


Các nguyên tắc thiết kế Agile

8. Các quy trình linh hoạt thúc đẩy phát triển bền vững. Các nhà tài trợ, nhà phát triển, và người dùng có thể duy trì một nhịp độ liên tục không giới hạn.
9. Liên tục quan tâm đến các kỹ thuật và thiết kế tốt để gia tăng sự linh hoạt.
10. Sự đơn giản – nghệ thuật tối đa hóa lượng công việc chưa xong – là căn bản.
11. Các kiến trúc tốt nhất, yêu cầu tốt nhất, và thiết kế tốt nhất sẽ được làm ra bởi các nhóm tự tổ chức.
12. Đội phát triển sẽ thường xuyên suy nghĩ về việc làm sao để trở nên hiệu quả hơn, sau đó họ sẽ điều chỉnh và thay đổi các hành vi của mình cho phù hợp.



Nội dung



4. Các mẫu thiết kế

5. Thiết kế giao diện người sử dụng

6. Thiết kế dữ liệu



Mẫu thiết kế

- Một mẫu thiết kế là một mô tả, một cách tái sử dụng kiến thức trừu tượng về:
 - Một vấn đề
 - Và bản chất giải pháp của nó.
- Mẫu thiết kế (Design Pattern) chủ yếu được dựa theo các nguyên tắc:
 - Lập trình hướng đến các interface và tính trừu tượng (program to an interface not implementation)
 - Ưu tiên object composite hơn là thừa kế

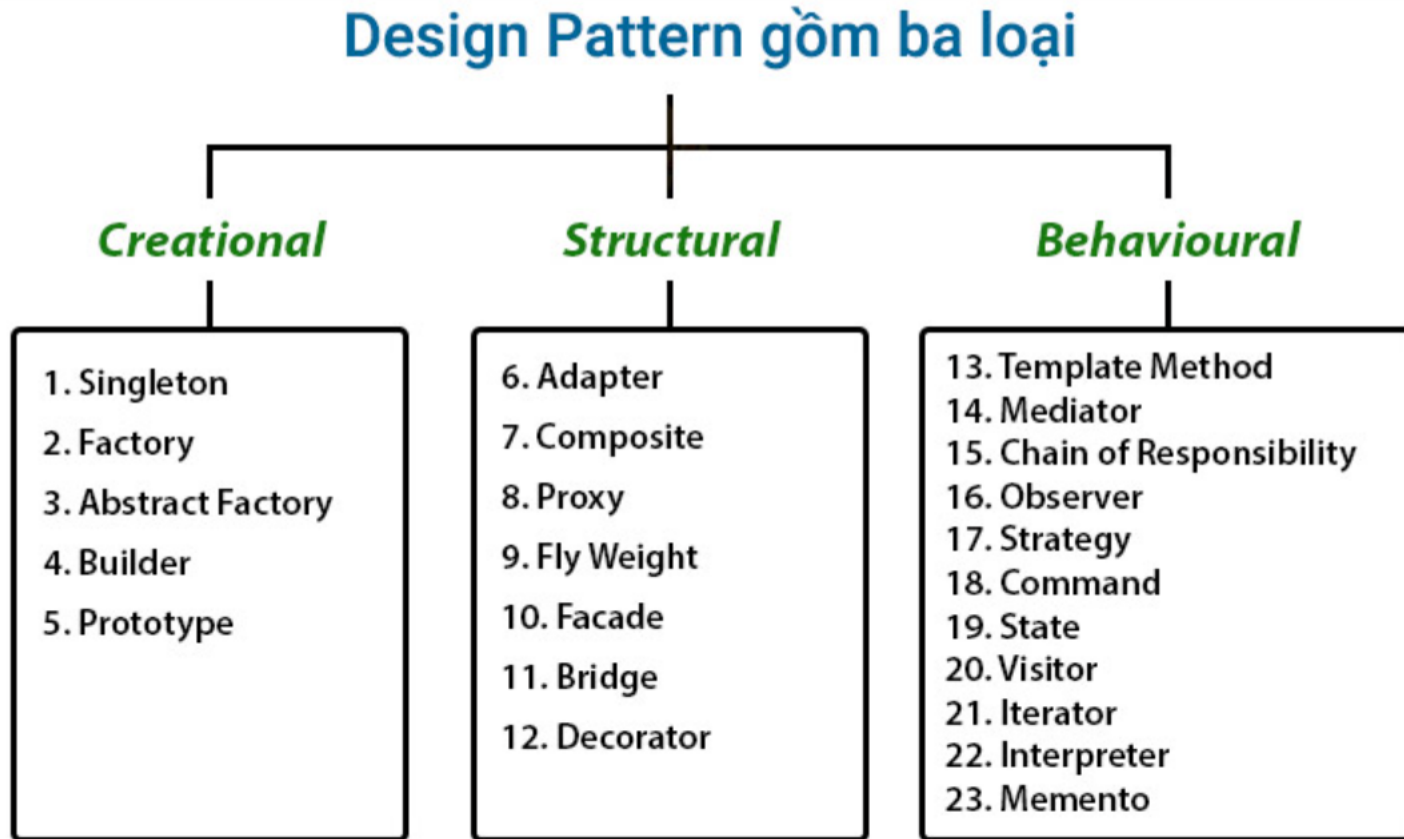


Phân loại mẫu thiết kế

- Design Patterns: Elements of Reusable Object Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, 1995 (Gang of Four)
- Creational Patterns
 - Abstract Factory, Builder, Factory Method, Prototype, Singleton
- Structural Patterns
 - Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
- Behavioral Patterns
 - Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor



Phân loại mẫu thiết kế





Các thành phần của một mẫu thiết kế

1. Tên mẫu thiết kế

- Là một tên gọi có ý nghĩa, ví dụ: Observer

2. Miêu tả vấn đề

3. Miêu tả giải pháp

- Không phải là một thiết kế cụ thể mà là một khuôn mẫu cho một giải pháp thiết kế có thể được khởi tạo cho nhiều cách khác nhau

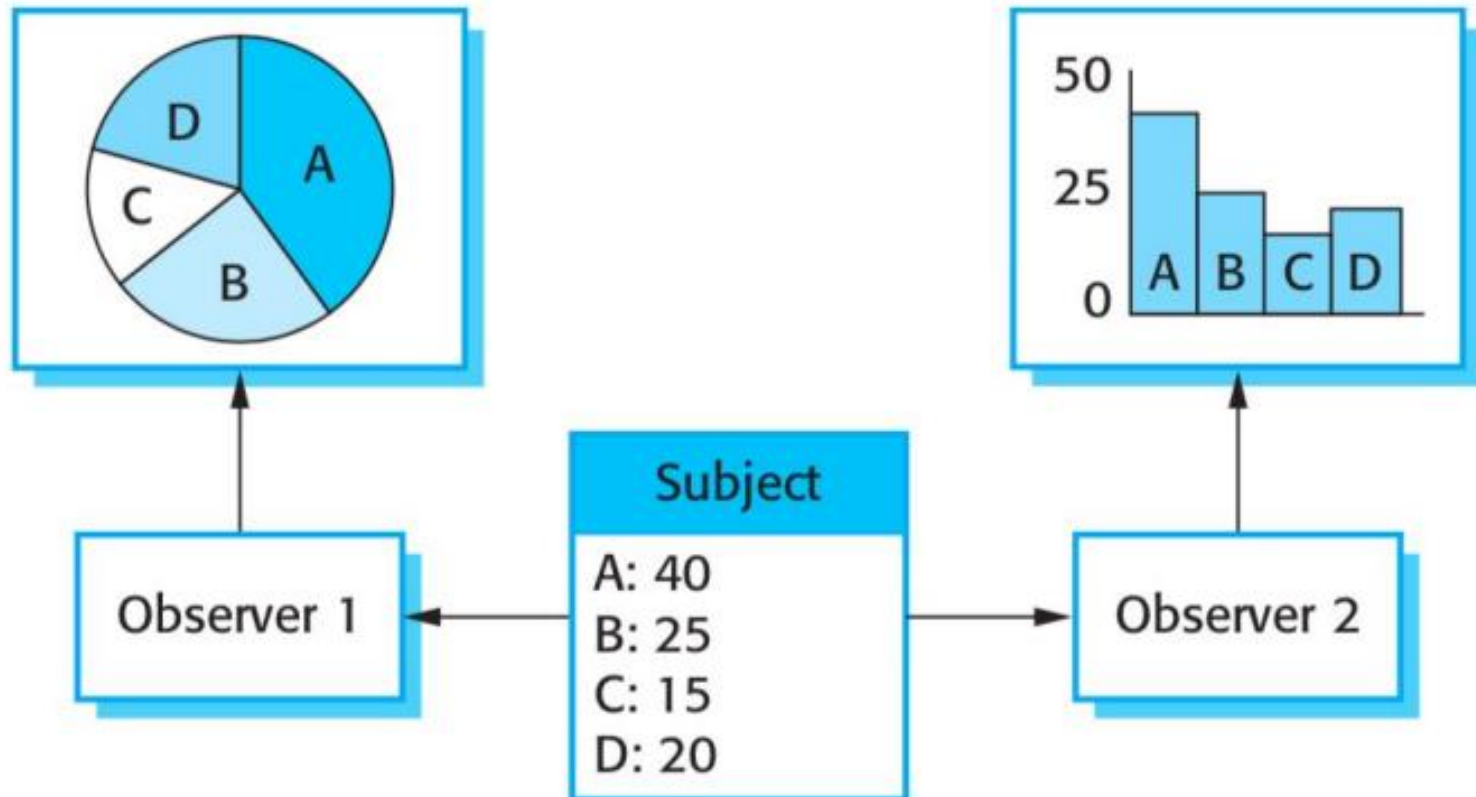
4. Các tác động

- Những kết quả và sự đánh đổi trong việc áp dụng mẫu.



Mẫu Observer

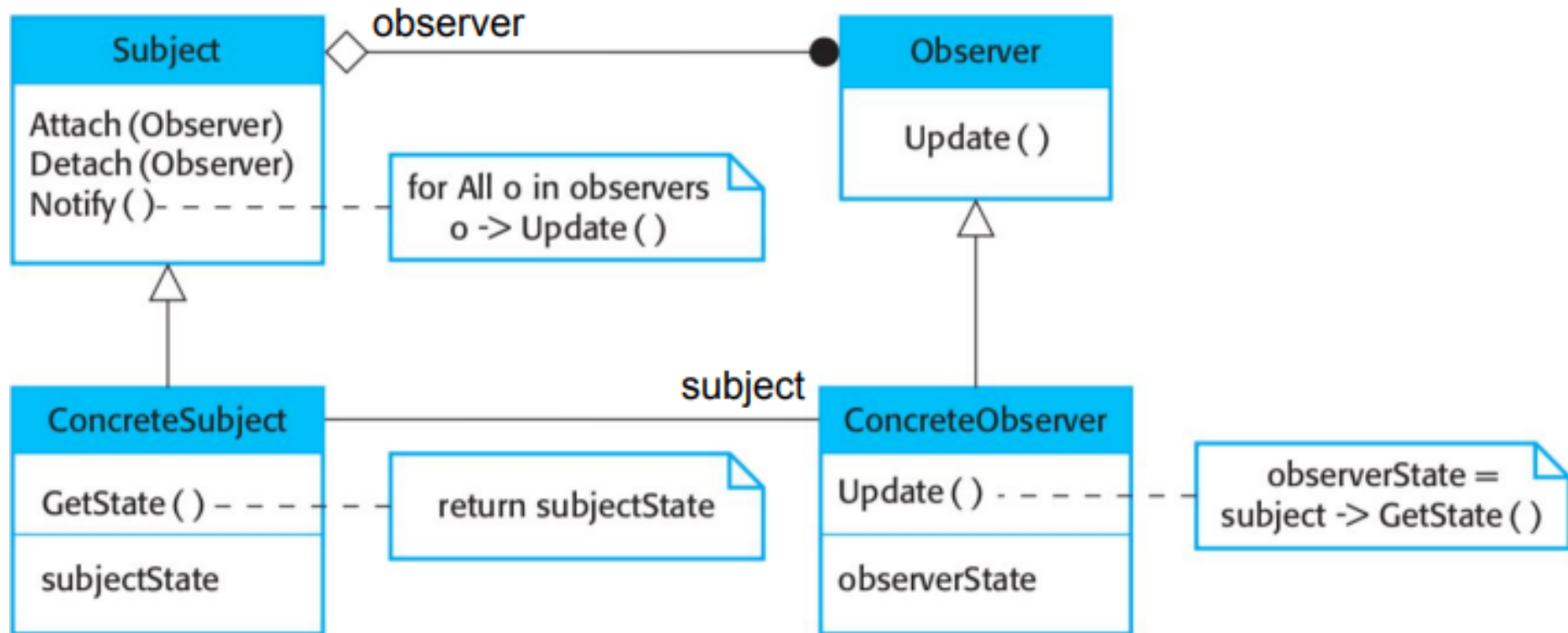
- Miêu tả vấn đề





Mẫu Observer

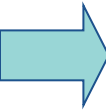
- Giải pháp





Nội dung

4. Các mẫu thiết kế



5. Thiết kế giao diện người sử dụng

6. Thiết kế dữ liệu



Nhân tố con người trong thiết kế giao diện

- Trí nhớ ngắn hạn có hạn
 - Người ta có thể nhớ nhanh khoảng 7 thông tin.
 - Nếu bạn đưa ra nhiều hơn như vậy, người ta sẽ dễ nhầm lẫn.
- Nhầm lẫn là chuyện thường
 - Khi người dùng nhầm lẫn và hệ thống chạy sai, những cảnh báo và thông báo không đúng mức có thể làm người dùng căng thẳng hơn và càng dễ nhầm lẫn hơn.
- Mỗi người có năng lực cao thấp khác nhau
 - Người thiết kế không nên chỉ thiết kế cho năng lực của chính mình.
- Mỗi người hợp với những kiểu tương tác khác nhau
 - Người này thích hình ảnh, người khác thích chữ.



Các nguyên tắc chính thiết kế UI

- Thiết kế UI phải xét đến nhu cầu, kinh nghiệm và năng lực của người dùng hệ thống.
- Người thiết kế cần
 - nhận thức được các hạn chế về vật lý và tâm lý của con người
 - Ví dụ giới hạn của trí nhớ ngắn hạn
 - nhận thức rằng con người ai cũng có thể nhầm lẫn.
- Các nguyên tắc thiết kế UI đóng vai trò nền tảng cho các thiết kế giao diện
 - Tuy rằng không phải tất cả các nguyên tắc đều áp dụng được cho tất cả các thiết kế.



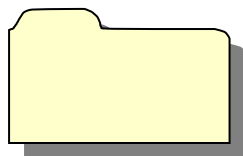
Các nguyên tắc thiết kế UI

Nguyên tắc	Mô tả
Quen thuộc với người dùng	Giao diện nên dùng các thuật ngữ và khái niệm rút ra từ kinh nghiệm của những người sẽ dùng hệ thống nhiều nhất.
Nhất quán	giao diện cần nhất quán sao cho các thao tác gần giống nhau có thể được kích hoạt theo cùng kiểu.
ngạc nhiên tối thiểu	Người dùng không bao giờ bị bất ngờ về hành vi của hệ thống.
khôi phục được	Giao diện nên có các cơ chế cho phép người dùng khôi phục lại tình trạng hoạt động bình thường sau khi gặp lỗi.
hướng dẫn người dùng	Giao diện nên có phản hồi có nghĩa khi xảy ra lỗi và cung cấp các tiện ích trợ giúp theo ngữ cảnh.
người dùng đa dạng	Giao diện nên cung cấp các tiện ích tương tác thích hợp cho các loại người dùng hệ thống khác nhau.



Các vấn đề khi thiết kế UI

- Hai vấn đề cần xem xét
 - Người dùng cung cấp thông tin cho hệ thống bằng cách nào?
 - Hệ thống nên trình bày thông tin (output) cho người dùng như thế nào?





Các kiểu tương tác vào dữ liệu

- Thao tác trực tiếp – Direct manipulation
- Chọn lựa bằng menu – Menu selection
- Điền form – Form fill-in
- Dòng lệnh – Command language
- Ngôn ngữ tự nhiên – a Natural language



WolframAlpha™ computational knowledge engine

Who is Vo Nguyen Giap?

Input interpretation: **Võ Nguyên Giáp (military)** *Mathematica form*

Basic information:

full name	Võ Nguyên Giáp
date of birth	August 25, 1911 (age: 98 years)
place of birth	Vietnam

Timeline:

Võ Nguyên Giáp

1900 1925 1950 1975 2000

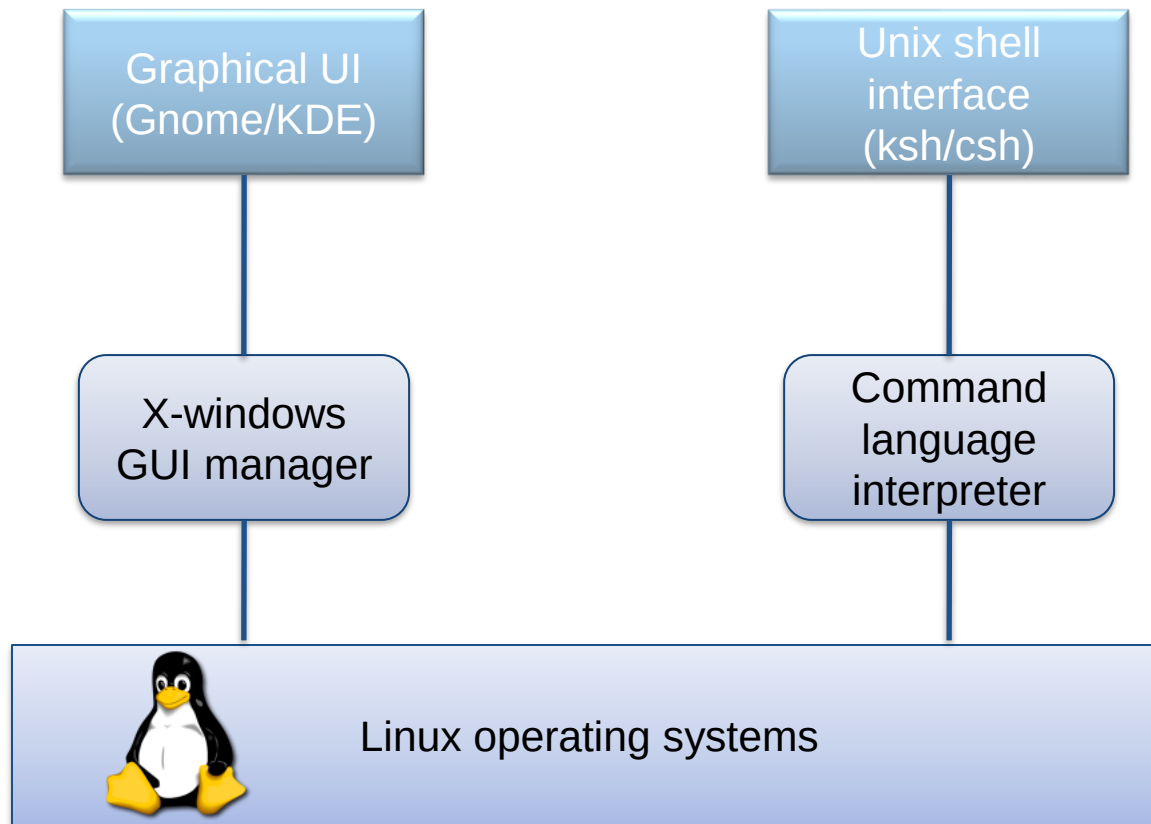


Các kiểu tương tác vào dữ liệu

Kiểu tương tác	Ưu điểm chính	Nhược điểm chính	Ví dụ ứng dụng
Direct manipulation	Tương tác nhanh và trực quan Dễ học	Có thể khó cài đặt. Chỉ thích hợp khi có ẩn dụ hình ảnh cho các tác vụ và đối tượng.	Trò chơi điện tử Các hệ thống dùng drag-n-drop
Menu selection	Tránh lỗi người dùng Không phải gõ nhiều	Chậm chạp đối với người dùng nhiều kinh nghiệm. Có thể phức tạp nếu có nhiều lựa chọn menu.	Đa số các hệ thống thông dụng
Form fill-in	Nhập dữ liệu đơn giản Dễ học Kiểm tra được	Tốn không gian màn hình. Gây rắc rối khi các lựa chọn của người dùng không khớp với các trường của form.	Khai thuế, xử lý nợ cá nhân
Command language	mạnh và linh động	Khó học xử lý lỗi kém	hệ điều hành
Natural language	Đáp ứng được người dùng không chuyên Dễ mở rộng	Cần gõ nhiều. Các hệ thống hiểu ngôn ngữ tự nhiên không đáng tin cậy.	Information retrieval systems



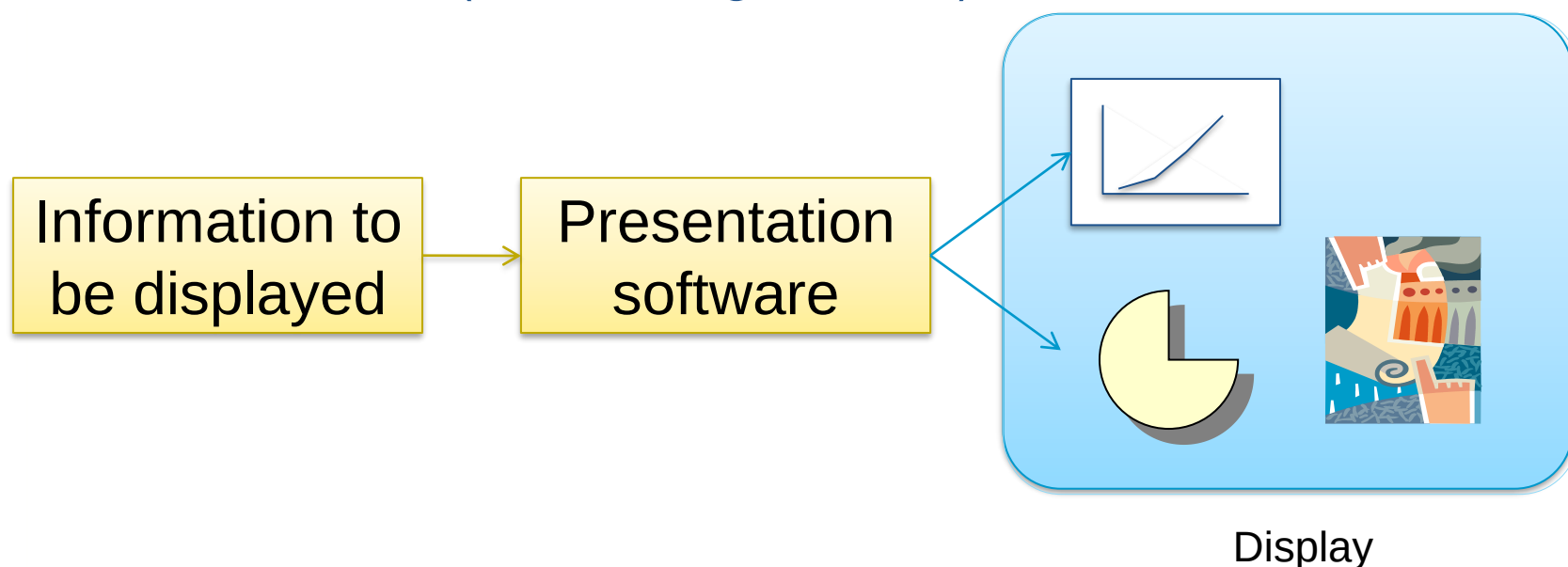
Multiple user interfaces





Biểu diễn thông tin ra (output)

- Trình bày thông tin hệ thống như thế nào cho người dùng.
- Thông tin có thể được trình bày trực tiếp (ví dụ text trong một trình soạn thảo) hoặc được biến đổi thành một dạng biểu diễn khác (ví dụ dạng đồ họa)





Biểu diễn thông tin ra (output)

- Thông tin tĩnh
 - Khởi tạo ở đầu session. Không thay đổi trong suốt session.
- Thông tin động
 - Thay đổi trong session và phải được thông báo với người dùng.



Các nhân tố khi hiển thị thông tin

- Người dùng quan tâm đến con số chính xác hay mối tương quan giữa các số liệu?
- Các giá trị của thông tin thay đổi nhanh chậm ra sao? Có phải lập tức thông báo khi có thay đổi?
- Người dùng có phải hành động để phản ứng với một thay đổi?



Các thông báo lỗi

- Thiết kế thông báo lỗi là việc rất quan trọng
 - Các thông báo lỗi kém có thể dẫn đến việc người dùng không chấp nhận sử dụng một hệ thống.
- Các thông báo nên lịch sự, ngắn gọn, nhất quán và mang tính xây dựng.
- Nên xét đến background và kinh nghiệm của người dùng như là nhân tố quyết định khi thiết kế các thông báo lỗi.



Các nhân tố khi thiết kế nội dung các thông điệp

Nhân tố	Miêu tả
Ngữ cảnh	Các thông điệp do hệ thống tạo ra cần phản ánh ngữ cảnh sử dụng hiện tại bất cứ khi nào có thể. Hệ thống cần cố gắng nhận biết được người dùng đang làm gì và nên tạo các thông điệp có liên quan đến hoạt động hiện tại của họ.
Kinh nghiệm	Khi người dùng quen dần với một hệ thống, họ sẽ trở nên khó chịu bởi các thông điệp dài 'có ý nghĩa'. Tuy nhiên, những người mới dùng lại thấy những thông báo ngắn gọn khó hiểu. Bạn nên cung cấp cả hai loại thông điệp và để cho người dùng kiểm soát mức độ xúc tích của các thông báo.
Trình độ kĩ năng	Các thông điệp nên được điều chỉnh theo trình độ kĩ năng cũng như kinh nghiệm của người dùng. Thông điệp dành cho các loại người dùng khác nhau có thể được diễn đạt theo các cách khác nhau tùy theo bộ thuật ngữ quen thuộc với người đọc.
Phong cách	Các thông điệp nên mang tính tích cực hơn là tiêu cực, nên dùng thể chủ động hơn là bị động, không bao giờ nên có giọng xúc phạm hoặc cố pha trò.
Văn hóa	Nếu có thể, người thiết kế nên là người quen thuộc với văn hóa của đất nước nơi hệ thống sẽ được bán. Có những khác biệt văn hóa rõ rệt giữa châu Âu, châu Á và châu Mỹ. Một thông điệp phù hợp với nền văn hóa này có thể không chấp nhận được ở một nền văn hóa khác.



Lỗi người dùng

- Giả sử một y tá đang cần tìm hồ sơ của một bệnh nhân nhưng lại gõ sai tên của bệnh nhân này.

Mời nhập tên của bệnh nhân vào hộp bên dưới rồi nhấn OK

Tên bệnh nhân

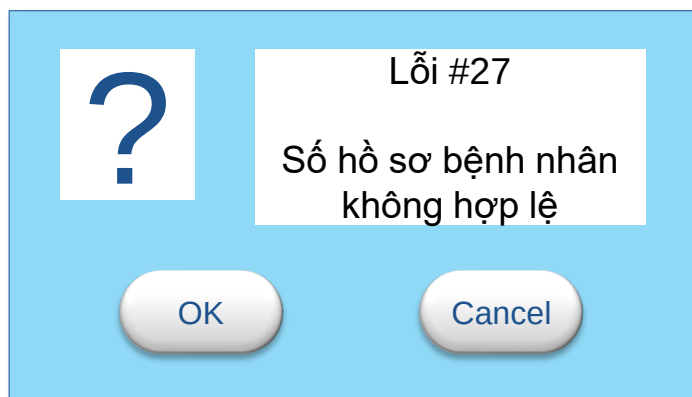
MacDonald, R.

OK Cancel

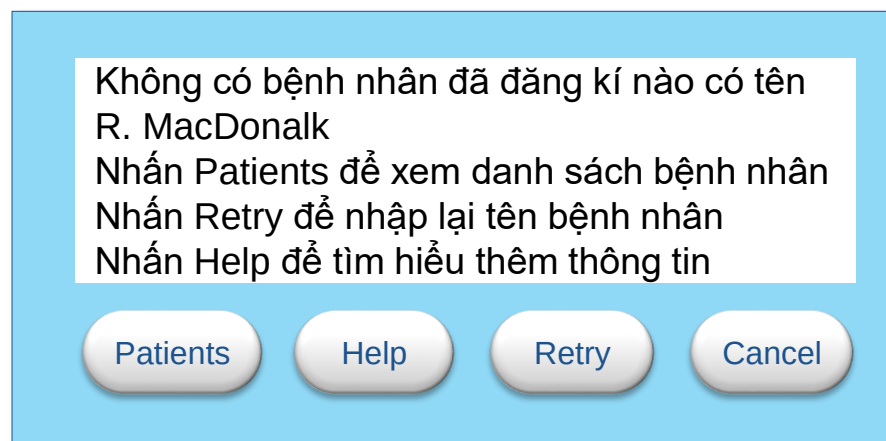


Thiết kế thông điệp tốt và tồi

Thông báo lỗi kiểu hệ thống



Thông báo lỗi kiểu người dùng



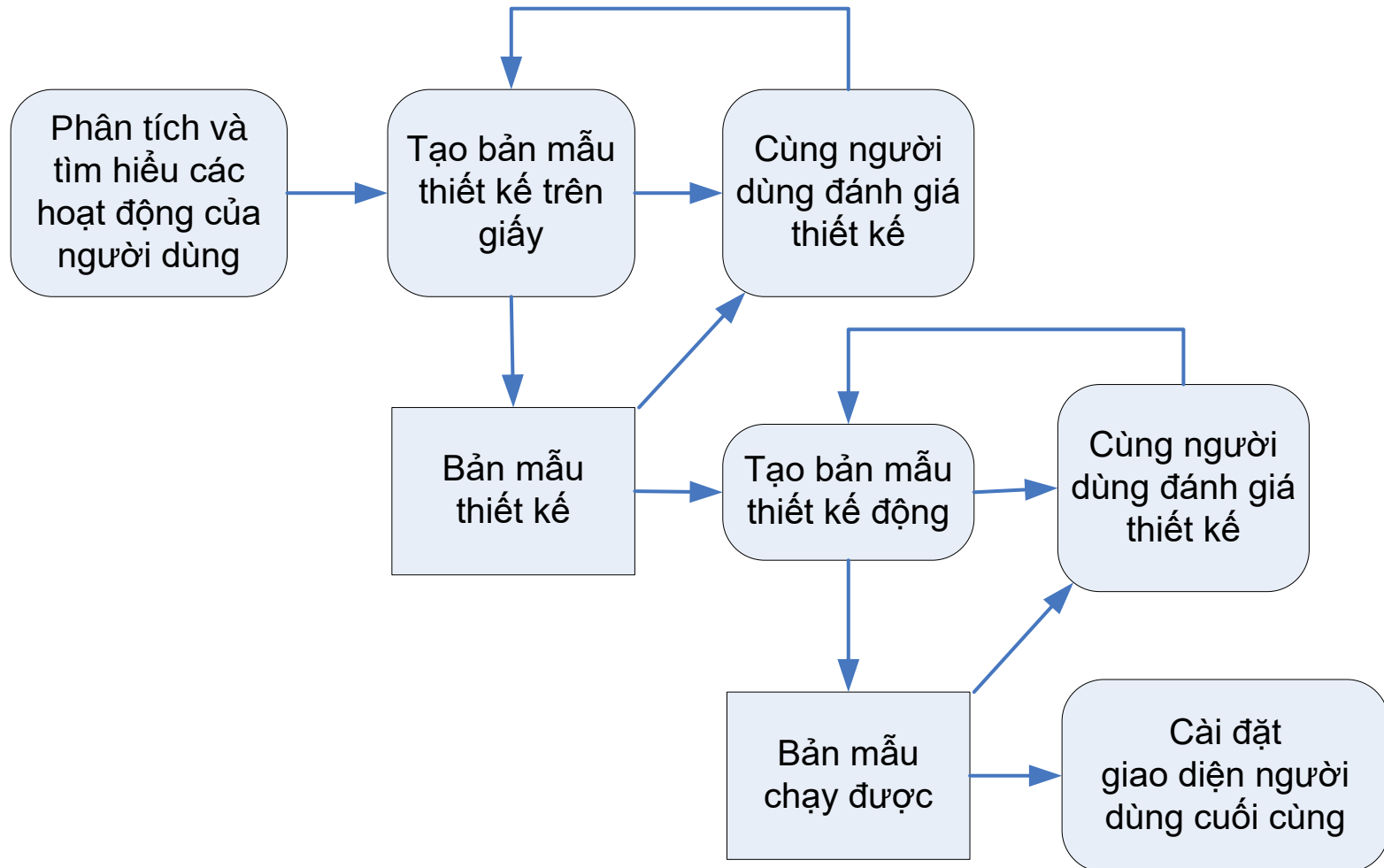


Quy trình thiết kế UI

- Thiết kế UI là một quy trình lặp đi lặp lại với sự liên lạc chặt chẽ giữa người dùng và người thiết kế.
- Ba hoạt động chính trong quy trình:
 - **User analysis:** Tìm hiểu người dùng sẽ làm gì với hệ thống;
 - **System prototyping:** phát triển một loạt các bản mẫu để thử nghiệm;
 - **Interface evaluation:** thử nghiệm các bản mẫu cùng với người dùng.



Quy trình thiết kế





Phân tích người dùng

- Nếu bạn không hiểu người dùng muốn làm gì với một hệ thống, bạn khó có thể thiết kế một giao diện hiệu quả.
- Các phân tích người dùng phải được mô tả theo cách mà cả người dùng lẫn người thiết kế đều có thể hiểu được.
- Các kịch bản mà bạn dùng để miêu tả các tình huống sử dụng điển hình là một cách để mô tả các phân tích này.



Các kỹ thuật phân tích người dùng

- Phân tích tác vụ - Task analysis
 - Mô hình hóa các bước trong việc hoàn thành một tác vụ.
- Phỏng vấn và câu hỏi điều tra (questionnaire)
 - Hỏi người dùng về công việc của họ.
- Văn hóa
 - Quan sát người dùng khi họ đang làm việc.



Bản mẫu thiết kế-UI

- Mục tiêu của bản mẫu là cho phép người dùng có được trải nghiệm trực tiếp đối với giao diện.
- Nếu không có trải nghiệm trực tiếp, không thể đánh giá khả năng sử dụng một giao diện.
- Quy trình làm bản mẫu có thể có 2 bước:
 - Lúc đầu, có thể dùng bản mẫu trên giấy;
 - Sau đó tinh chỉnh thiết kế, và phát triển các bản mẫu tự động hóa với độ phức tạp ngày càng tăng.



Bản mẫu thiết kế-trên giấy

- Dùng các bản vẽ nháp về UI để duyệt qua các kịch bản.
- Dùng storyboard để trình bày một chuỗi tương tác với hệ thống.
- Bản mẫu giấy là cách thu thập phản ứng của người dùng đối với một đề xuất thiết kế.



Các kỹ thuật làm bản mẫu

- Script-driven prototyping
 - Viết một loạt các script và screen bằng các công cụ. Ví dụ như figma...
 - Khi người dùng tương tác, screen sẽ chuyển sang hiển thị màn hình tiếp theo.
- Visual programming
 - Dùng một ngôn ngữ chuyên dụng cho phát triển nhanh như Visual Basic.
- Internet-based prototyping
 - Dùng một trình duyệt web browser và các script kèm theo.



Các kỹ thuật làm bản mẫu

- Hướng dẫn sử dụng FIGMA



Nội dung

- 4. Các mẫu thiết kế
- 5. Thiết kế giao diện người sử dụng
- 6. Thiết kế dữ liệu



Thiết kế dữ liệu

- Mục tiêu: Mô tả cách thức tổ chức lưu trữ dữ liệu của phần mềm trong máy tính
- Ba yêu cầu:
 - Thiết kế dữ liệu với tính đúng đắn
 - Thiết kế dữ liệu với tính tiến hóa
 - Thiết kế dữ liệu với tính hiệu quả về mặt truy xuất và lưu trữ



Thiết kế dữ liệu

- Mô hình hóa dữ liệu với mô hình dữ liệu quan hệ:
 - Có hai cách tiếp cận để mô hình hóa dữ liệu:
 - Cách 1: Tiếp cận Top-Down
Vẽ mô hình thực thể liên kết và chuyển đổi thành sơ đồ logic trong mô hình quan hệ
 - Cách 2: Tiếp cận Bottom_up
Chuẩn hóa



Thảo luận

